

FILM ATIQUIE

필름아티크

Teamproject_매운 짬뽕팀



목차



- 프로젝트 기획

About Film Atique, Persona, Timeline

- 기획/구상

Typography & Color, Wireframe

- 웹사이트 제작

Code

- 후기

Review

Our Team



FILMATIQUE
CINEMAS

매운짬뽕팀 / 팀장

강민식

mobile 010-4565-9548
e-mail ste_reo@naver.com
git-hub <https://github.com/kms12191>



FILMATIQUE
CINEMAS

매운짬뽕팀 / 팀원

유영찬

mobile 010-7617-5850
e-mail sunhama2000@naver.com
git-hub <https://github.com/Y-youngchan>



FILMATIQUE
CINEMAS

매운짬뽕팀 / 팀원

김동환

mobile 010-2557-3564
e-mail iensud184@naver.com
git-hub <https://github.com/iensud184-max>



FILMATIQUE
CINEMAS

매운짬뽕팀 / 팀원

조예연

mobile 010-3930-1835
e-mail bomi1835@naver.com
git-hub <https://github.com/bomi6231835-jpg>

Film Atique



필름아티크는 저희가 만든 가상의 영화관 브랜드입니다.
그리고 또한 영화 관람권 디자인을 직접 제작하기도 하였습니다.
사용자가 실제 영화관처럼 편리하게 예매할 수 있는 서비스를 구현하면서,
실무 수준의 웹 개발 역량을 키우기 위해 만든 프로젝트입니다.



Persona

라이프 스타일

영화 굿즈 포스터를 모으는 것을 좋아하는 "20대 초반 김민아"입니다. 이벤트 페이지를 수시로 들어가며 새로운 소식이 올라왔는지 매일 확인합니다. 심지어 굿즈는 선착순으로 나눠주기 때문에 빠르게 확인을 하는 이유기도 합니다.

선택한 이유

필름아티크는 이벤트페이지가 메인화면에 있기 때문에 바로 들어가서 확인하는 것이 편함 심지어 무슨 이벤트를 하는지 메인페이지만 들어가도 확인이 가능하다.

김민희 (대학생)
2007년생 06월 10일 만19세



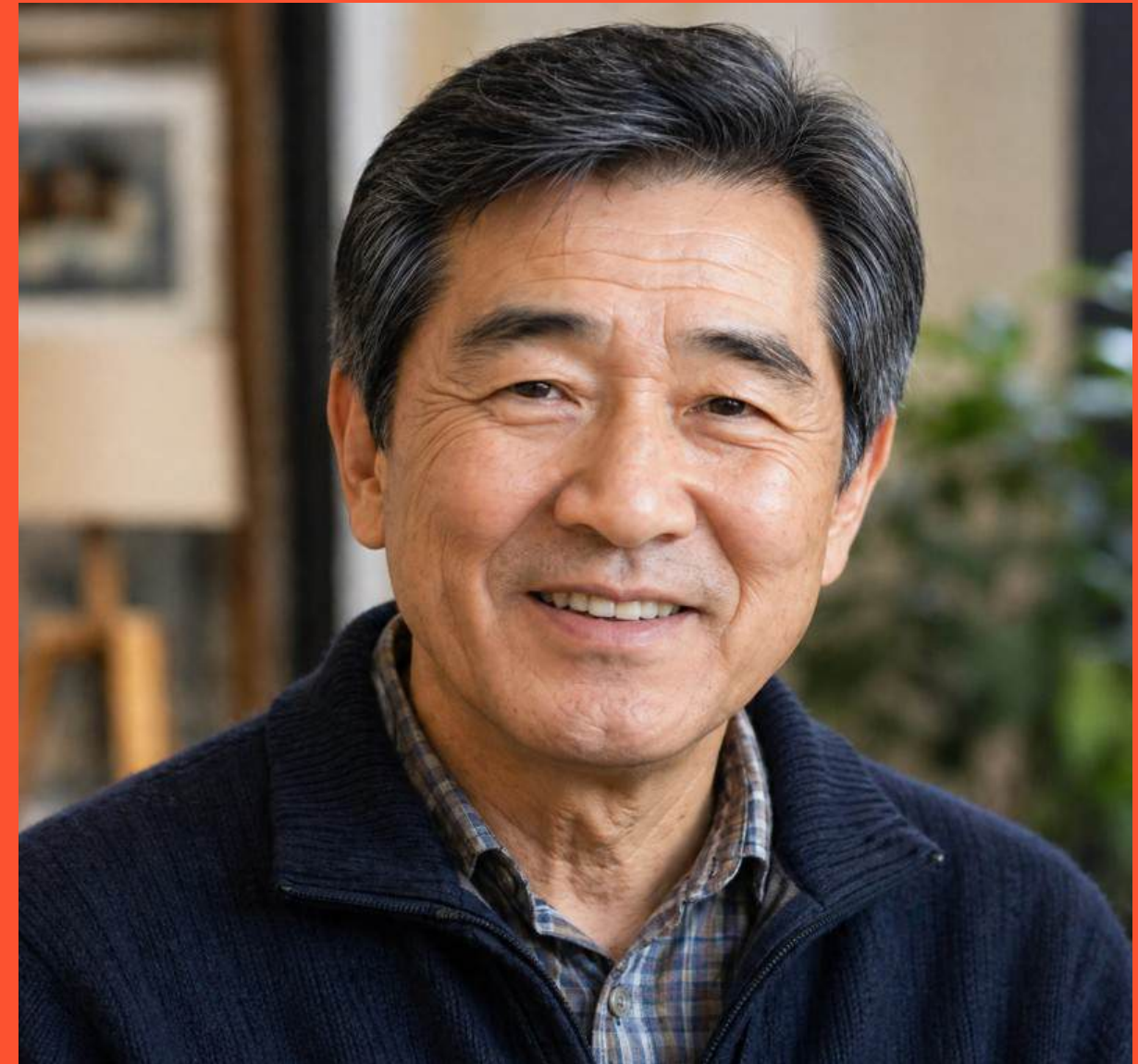
Persona

라이프 스타일

은퇴 후 여유로운 생활을 즐기며,
주말에는 취미로 영화 관람을 자주 합니다.
스마트폰 사용은 가능하지만 복잡한 기능보다는
단순하고 직관적인 서비스를 선호합니다.
가족이나 지인과 함께 시간을 보내는 것을 중요하게 생각합니다.

선택한 이유

화면이 직관적이고 글씨가 커서 사용하기 편리하다고 느꼈다.
복잡하지 않은 예매 과정 덕분에 빠르게 영화 예매가 가능했다.
필요한 정보가 한눈에 보여 별도의 검색 없이 쉽게 이용할 수 있었다.



김영수 (민물어부)
1965년생 07월 25일 만 60세

Persona

라이프 스타일

평일: 회사 → 운동 → 집에서 넷플릭스

주말: 브런치 카페, 영화 감상, 한강 산책

인스타는 과하지 않게 감성 사진 위주로 활동합니다.

여행은 국내로 조용하거나 분위기 좋은 곳으로 다닙니다.

영화나 애니메이션을 좋아합니다.

선택한 이유

전국 지점으로 영화관이 많고

조용히 혼자 영화보기를 좋아하는데 알맞은 영화관이다

홈페이지 예매를 자주 이용하는 편인데

각 지점 별 공지사항 보기 편하면서 깔끔하게 정리되어있고

마이페이지 예매 정보, 매점 구매 내역 등 한눈에 확인이 가능하며

전지점 예매가 가능해 여러지점 시간표나 상영중인 영화 확인이 빠르고

매점 및 굿즈 페이지에 상품 진열 또는 품질 확인이 확실하여 이용하는데 편

안함을 주어 필름아티크 영화관 브랜드 자주 애용한다.



한수아 (브랜드 마케팅팀 대리)

1996년생 03월 31일 만 30세

Persona

라이프 스타일

평일에는 업무가 매우 바빠 여가 시간이 거의 없습니다.
퇴근 후에는 피로도가 높기 때문에 복잡한 활동보다 간단하고 확실한 여가를 선호합니다.

주말에는 스트레스를 해소하기 위해 영화 관람을 자주 하며,
보통 토요일 저녁 또는 일요일 오후에 영화를 봅니다.

선택한 이유

영화 예매 서비스의 핵심 사용자가 빠르고 간편한 예매를 원하는
사용자이기 때문이다.
그중에서도 바쁜 직장인은 시간에 대한 민감도가 높고, 복잡한 과정 없이
즉시 예매를 완료하려는 성향이 강하다.

즉, 최소한의 클릭으로 빠르게 예매를 완료하는 경험을 설계하기 위해 가장
기준이 되는 사용자이다.



김지훈 (IT 회사 대리)
1992년생 10월 5일 만 33세

작업표

기간 : 04월06일 - 04월29일

[illegible]

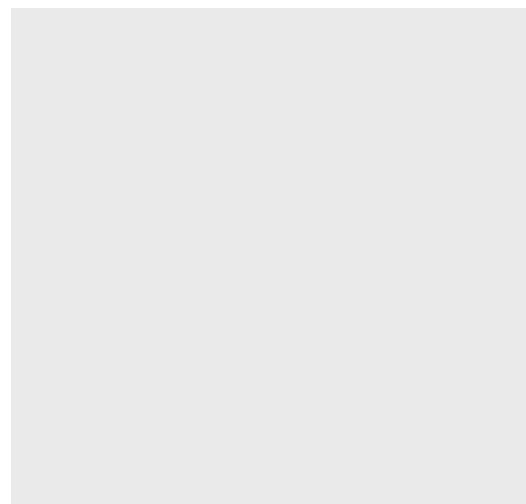
Typography & Color

Color

열정의 상징적인 색상인
붉은색을 많이 사용하였습니다.
사용자에게 눈에 확 띄는 컬러색으로
포인트를 주고 싶은 곳에 사용하였습니다.



#f0000f



#eaeaea



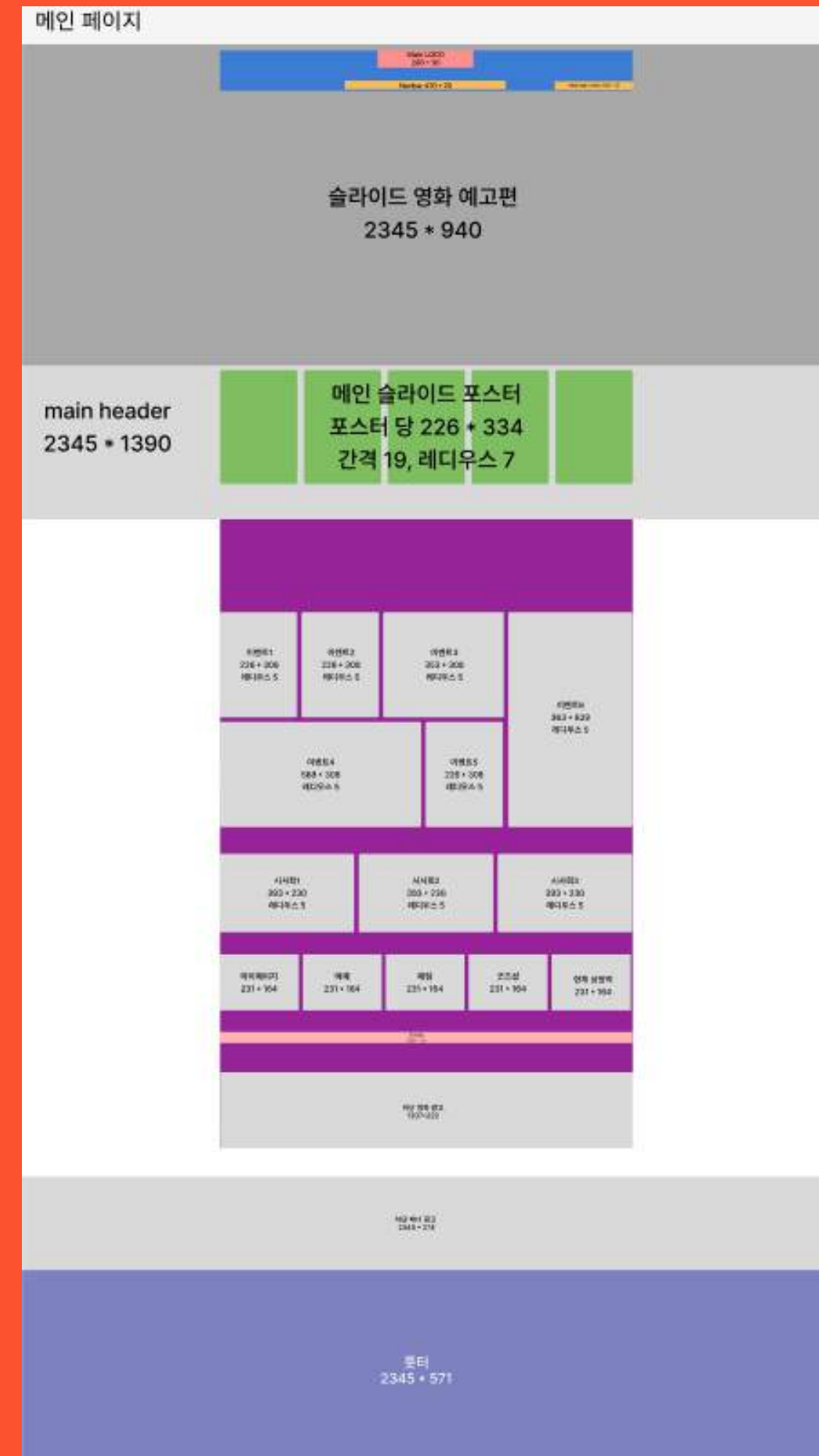
#444444

Typography

내용을 가독성 있게 볼 수 있는
'Franklin Gothic Medium',
'Arial Narrow', 'Arial', ' sans-serif '
서체를 활용하여 최대한 영화예매 사이트와
비슷한 느낌을 재현하려고 했습니다.

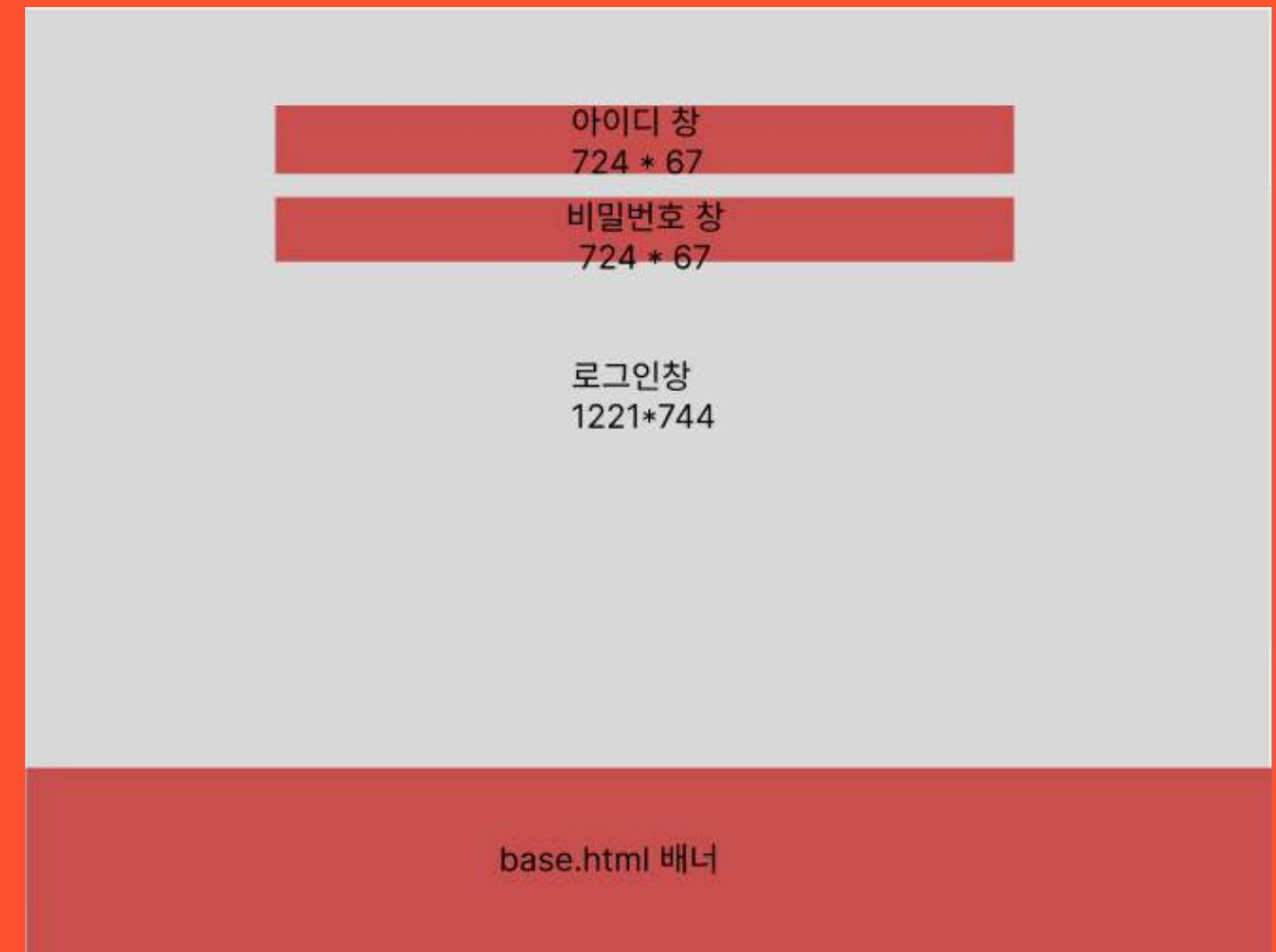
Franklin Gothic Medi
Arial

필름 아티크 메인페이지



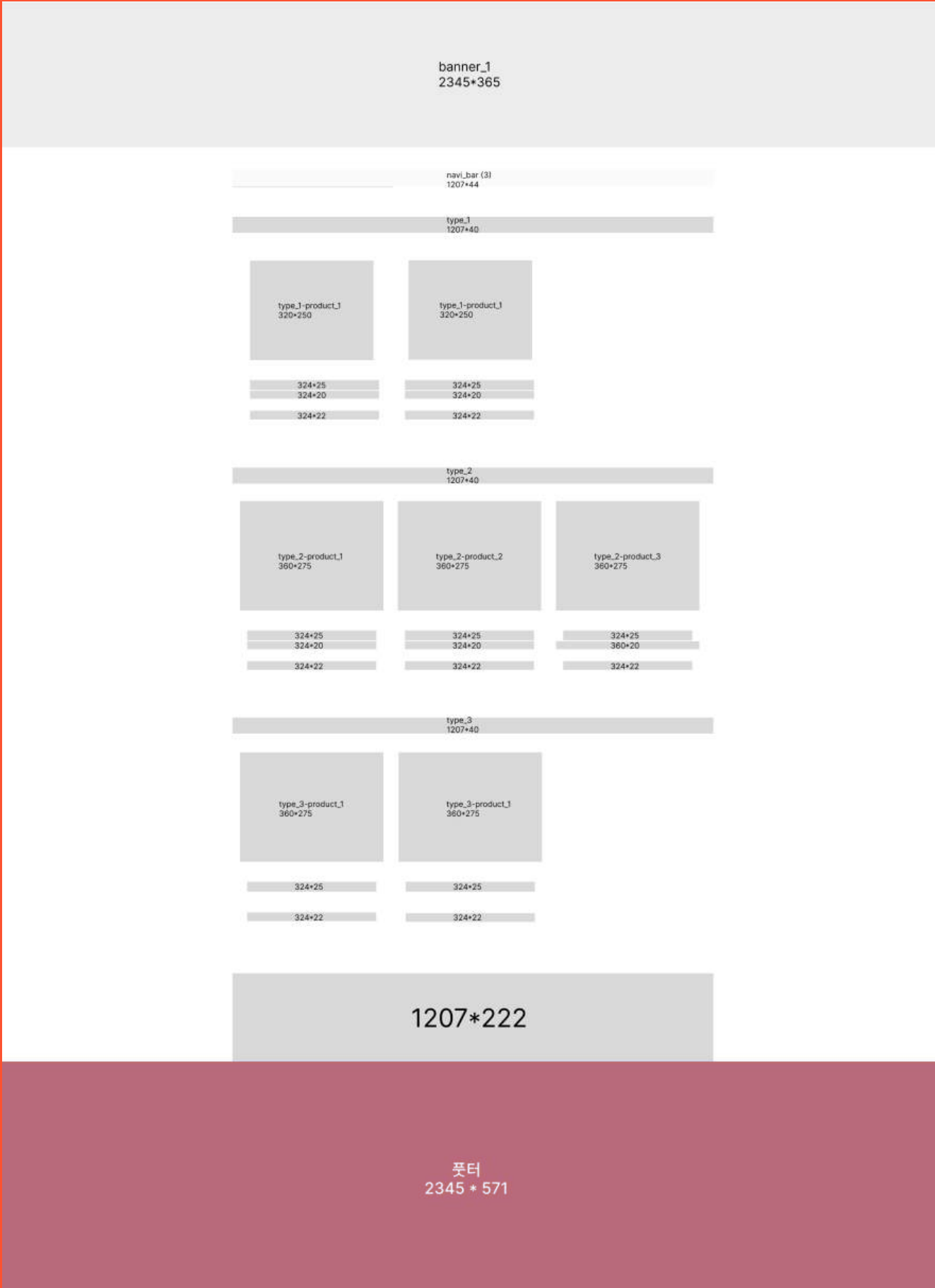
WireFrame

필름 아티크 서브페이지
(로그인 페이지)



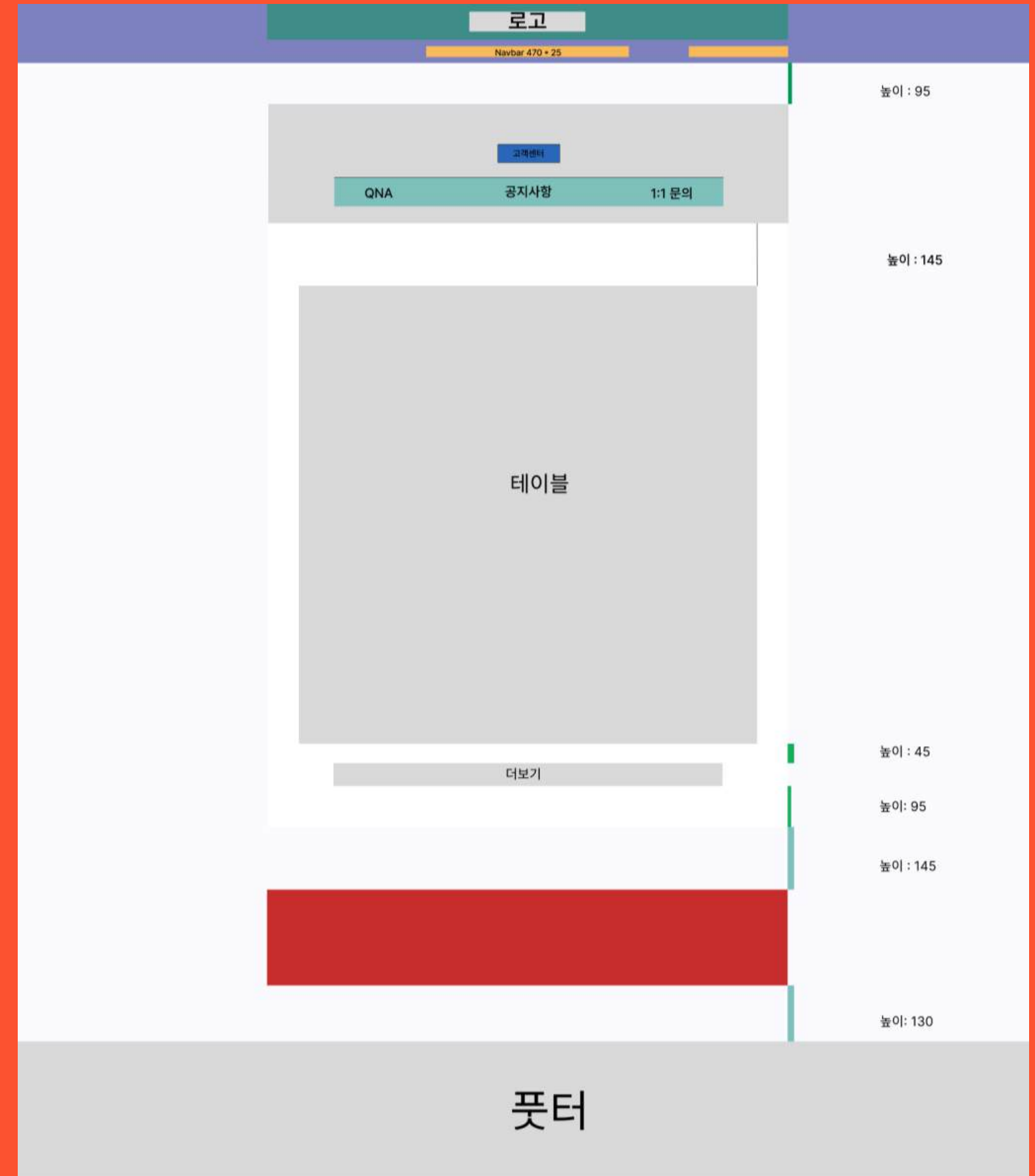
WireFrame

필름 아티크 서브페이지
(매점 & 굿즈샵 페이지)



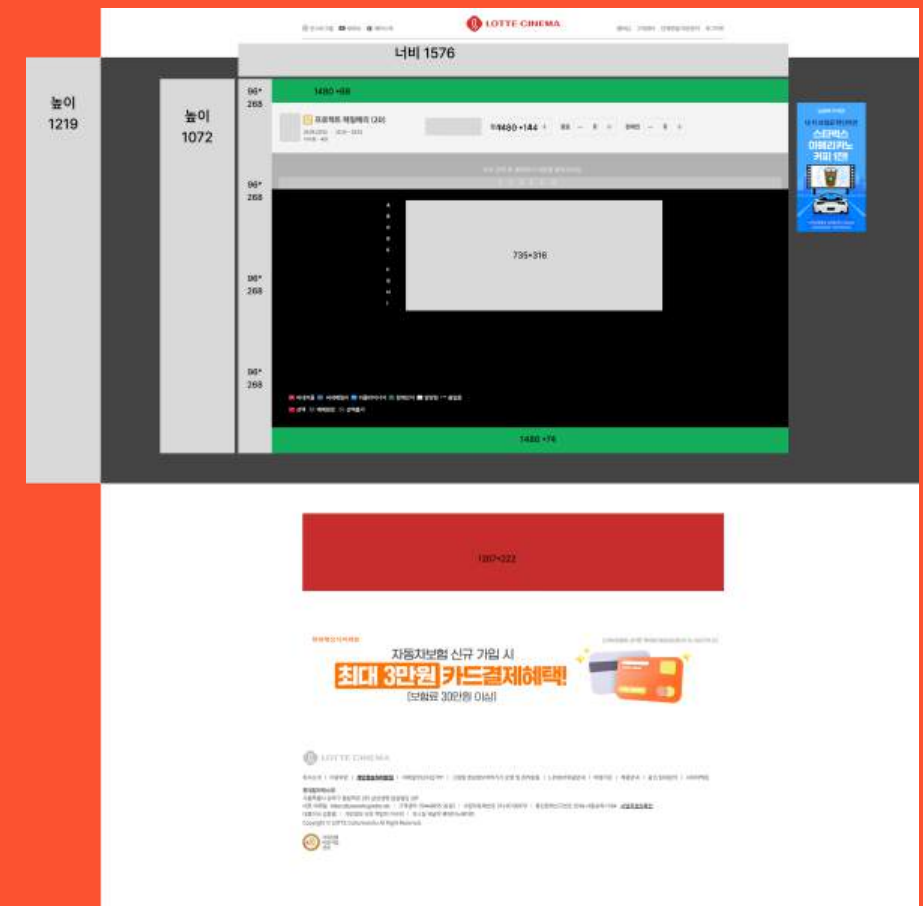
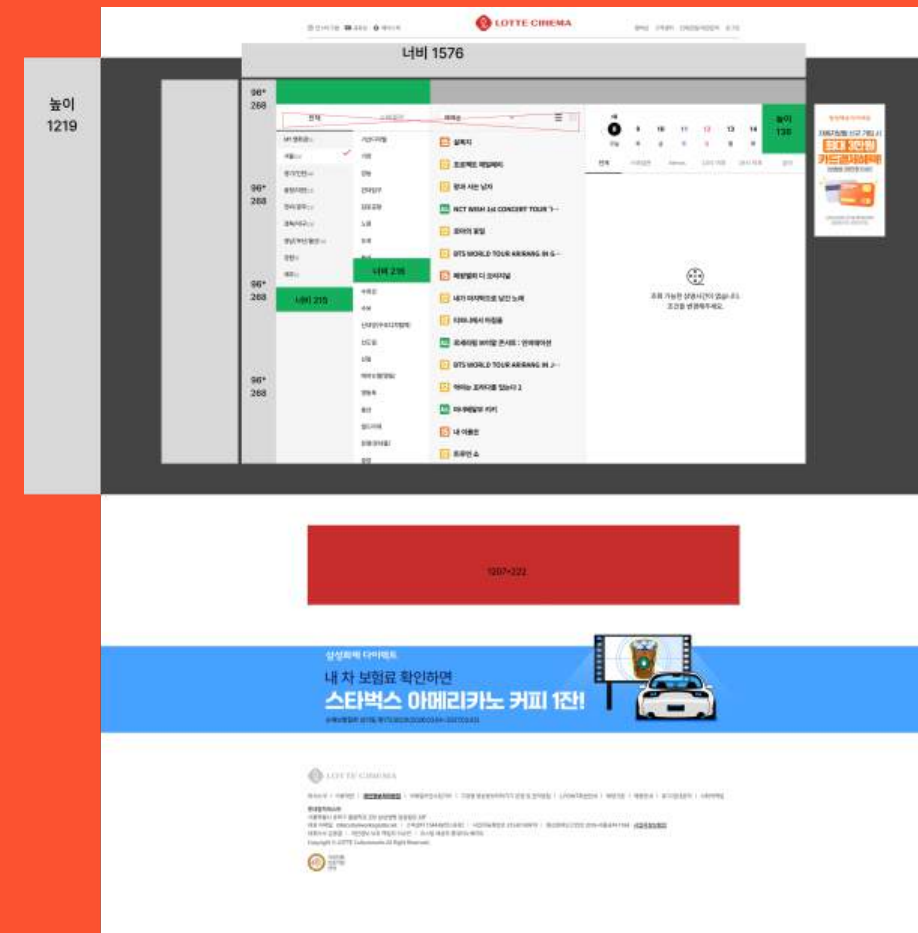
WireFrame

필름 아티크 서브페이지
(공지사항 페이지)



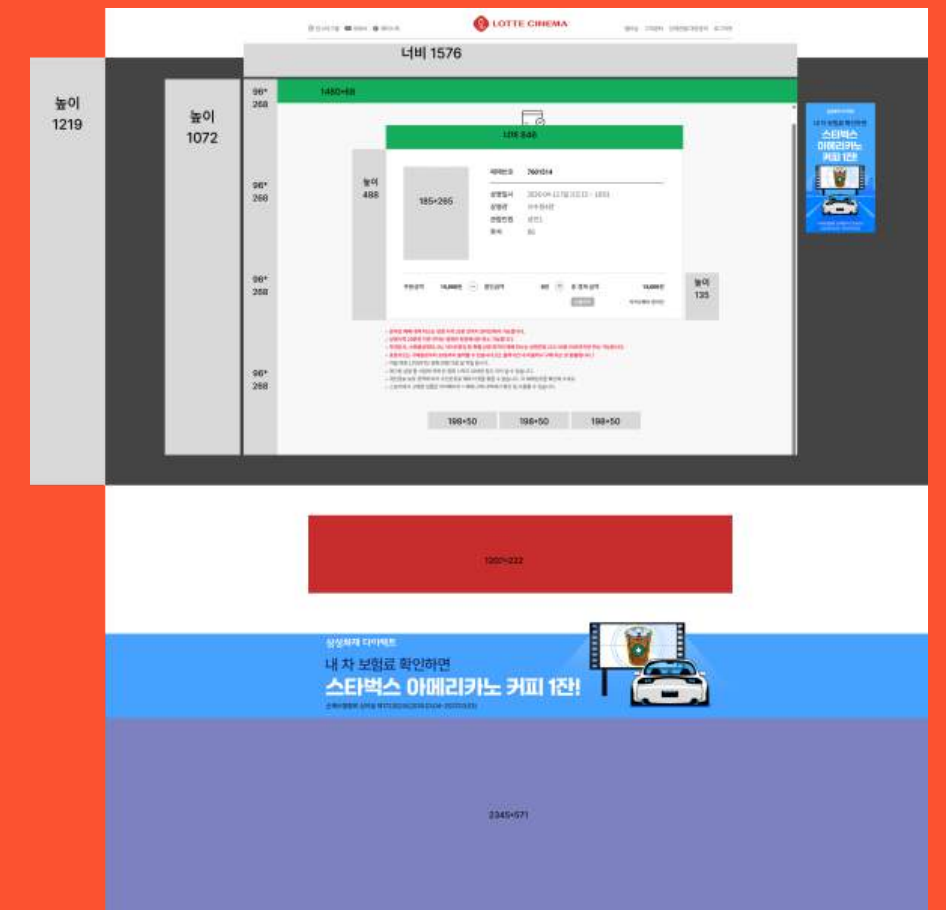
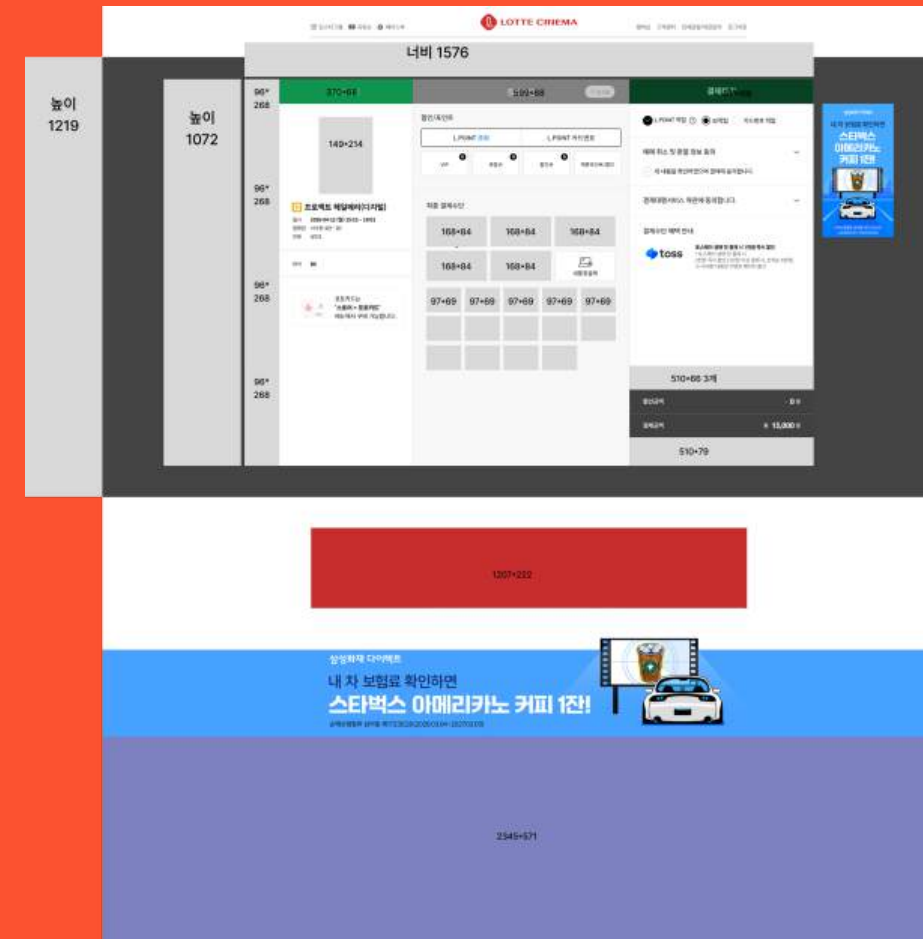
WireFrame

필름 아티크 서브페이지
(영화 예매 선택 페이지, 좌석선택페이지)



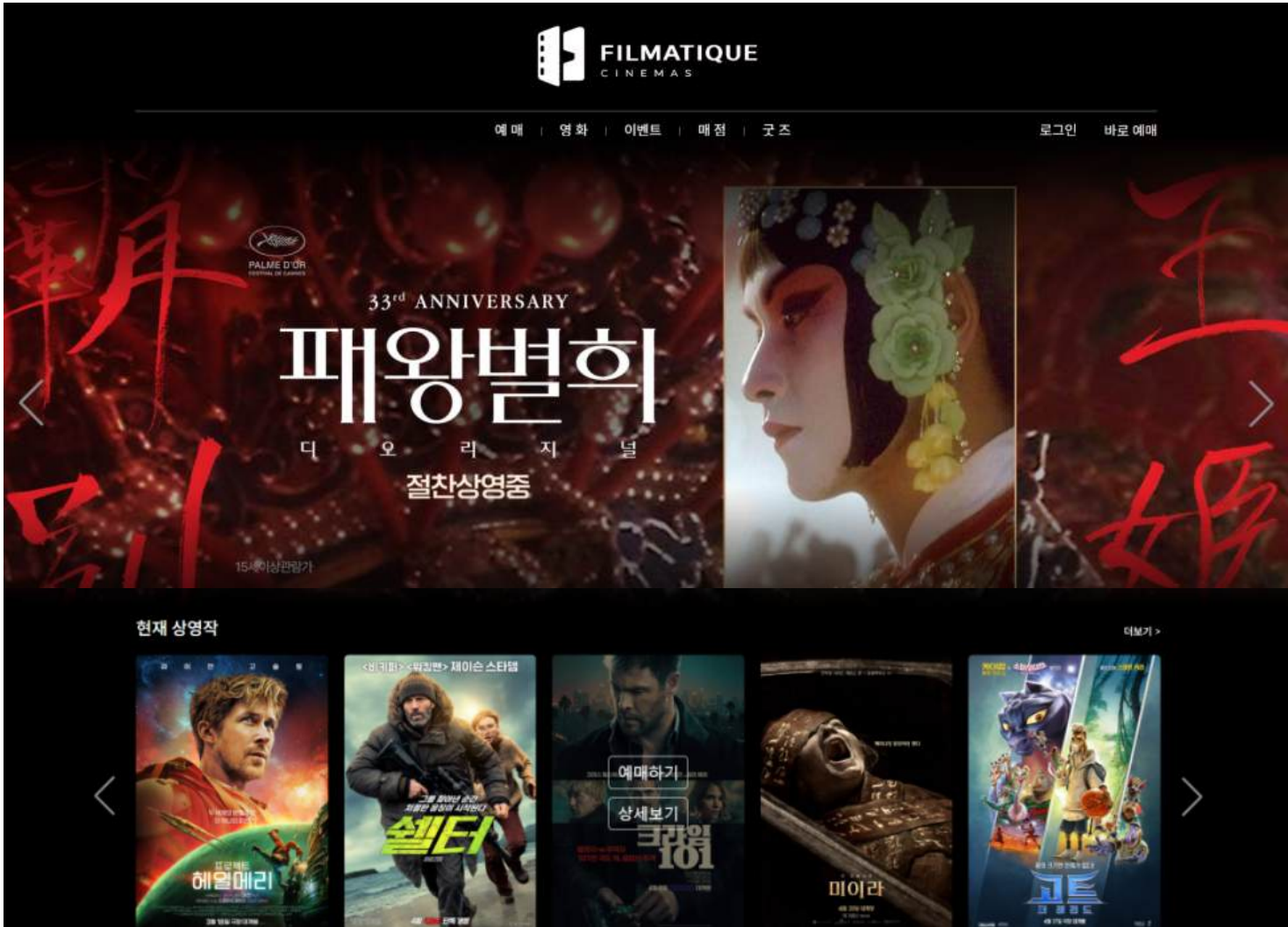
WireFrame

필름 아티크 서브페이지
(결제 수단 선택 페이지, 결제완료 페이지)



CODE (메인)

메인페이지(화면구현)



```
63 <!-- Swiper -->
64 <div class="trailer">
65   <div class="swiper mainSwiper">
66     <div class="swiper-wrapper">
67       {% for img in main_images %}
68       <div class="swiper-slide">
69         
70       </div>
71       {% endfor %}
72     </div>
73     <div class="swiper-button-next"></div>
74     <div class="swiper-button-prev"></div>
75   </div>
76 </div>

77
78
79 <div class="postertxt">
80   <p>현재 상영작</p>
81   <a href="{{ url_for('film.movie_list') }}">더보기 </a>
82 </div>
83 <!-- Swiper -->
84 <div class="poster">
85   <div class="swiper posterSwiper">
86     <div class="swiper-wrapper">
87       {% for movie in movies %}
88       <div class="swiper-slide">
89
90         
91
92         <div class="txtbox">
93           <a href="{{ url_for('film.booking', movie_id=movie.id) }}">예매하기</a>
94           <a href="/film/movie/{{ movie.tmbd_id }}">상세보기</a>
95         </div>
96       </div>
97       {% endfor %}
98     </div>
99   </div>
100 </div>
101 <div class="swiper-button-next"></div>
102 <div class="swiper-button-prev"></div>
103 </div>
104 </header>
```

swiper를 사용하여
슬라이더 형식의 느낌을 주었다.

background: linear-gradient를
사용하여 그라디언트를 사용하였다.

```
.headermain .topnav_right .topnav_reservation {
  padding: 8px 0;
}

.trailer {
  position: relative;
  margin-bottom: 10px;
}

.trailer::before {
  content: "";
  position: absolute;
  top: 0;
  left: 0;
  width: 100%;
  height: 200px;
  background: linear-gradient(to bottom, rgba(0, 0, 0, 1), rgba(0, 0, 0, 0));
  z-index: 2;
}

.trailer::after {
  content: "";
  position: absolute;
  bottom: 0;
  left: 0;
  width: 100%;
  height: 200px;
  background: linear-gradient(to top, rgba(0, 0, 0, 1), rgba(0, 0, 0, 0));
  z-index: 2;
}

.trailer .swiper {
  width: 100%;
  height: auto;
  position: relative;
  z-index: 1;
}

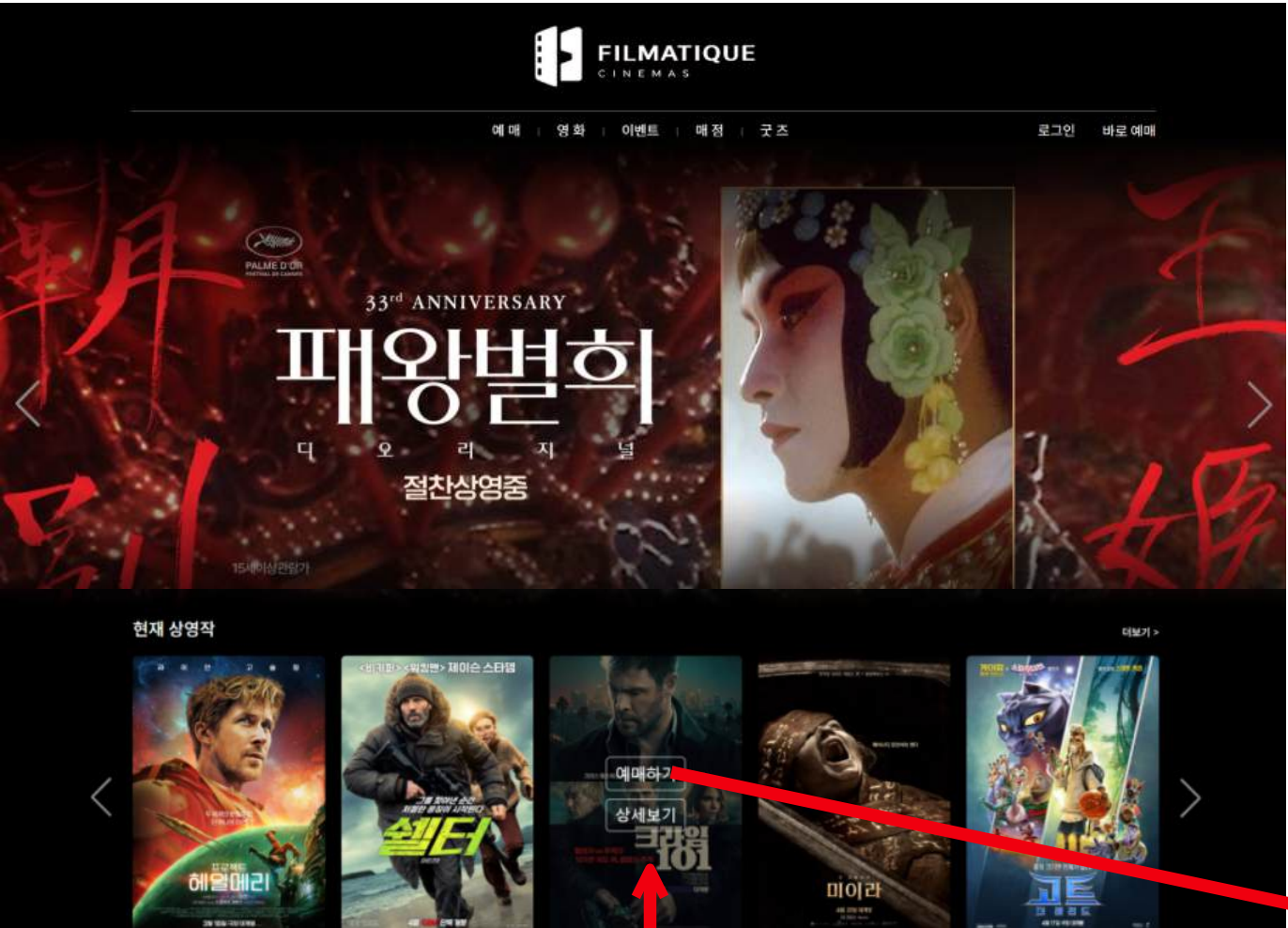
.trailer .swiper-slide {
  text-align: center;
  font-size: 18px;
  background: #000;
  display: flex;
  justify-content: center;
  align-items: center;
```

HTML

CSS

CODE (메인)

메인페이지(화면구현)



```
1 // Initialize Swiper
2 var swiper : Swiper = new Swiper(".mainSwiper", {
3   loop: true,
4   navigation: {
5     nextEl: ".mainSwiper .swiper-button-next",
6     prevEl: ".mainSwiper .swiper-button-prev",
7   },
8 },
9 );
10
11
12
13 var swiper : Swiper = new Swiper(".posterSwiper", {
14   slidesPerView: 5,
15   spaceBetween: 19,
16   loop: true,
17   navigation: {
18     nextEl: ".poster .swiper-button-next",
19     prevEl: ".poster .swiper-button-prev",
20   },
21 },
22 );
23
24 var swiper : Swiper = new Swiper(".noticeSwiper", {
25   direction: "vertical",
26   slidesPerView: "auto",
27   loop: true,
28   autoplay: {
29     delay: 2000,
30   },
31 },
32 );
```

Java Script

클릭 시 해당 영화가 선택된 상태로 예매 페이지 연결

	영화관	영화	날짜/시간							
01. 상영시간	서울	강남스트리트점	42 프로젝트 헤일메리	28 파	29 수	30 목	1 금	2 토	3 일	4 월
	경기/인천	가산디지털점	48 셀터	상영시간 없음						
	강원	견대점	59 리 크로닌의 미이라							
	충청/대전	용산점	48 크라임 101	작성선택						
	전라/광주	홍대점	44 고트: 더 레전드							
02. 인원/좌석	경북/대구	광고점	55 극장판 귀멸의 칼날: 무한성편							
	경남/부산	부천점	59 스크림 7							
	제주	동탄점	48 쇼생크 탈출							
		수원역점	59 노멀							
			48 트루먼 쇼							

```
<!-- Swiper -->
<div class="poster">
  <div class="swiper posterSwiper">
    <div class="swiper-wrapper">
      {% for movie in movies %}
      <div class="swiper-slide">
        
        <div class="txtbox">
          <a href="{{ url_for('film.booking', movie_id=movie.id) }}">예매하기</a>
          <a href="/film/movie/{{ movie.tmbd_id }}">상세보기</a>
        </div>
      </div>
      {% endfor %}
    </div>
    <div class="swiper-button-next"></div>
    <div class="swiper-button-prev"></div>
  </div>
</div>
```

영화 포스터 호버시

CODE (회원가입)

회원가입 페이지 화면구현

회원가입

아이디
비밀번호
비밀번호 확인
이름
이메일
전화번호
생년월일

☐ 회원이용약관 동의 (필수) [내용보기](#)
☐ 개인정보처리방침 동의 (필수) [내용보기](#)
☐ 이메일 수신 동의 (선택)

저장하기

Python

```
@bp.route('/signup', methods=['GET', 'POST'])
def signup():
    form = UserCreateForm()

    if request.method == 'POST' and form.validate_on_submit():
        existing_user = User.query.filter_by(userid=form.userid.data).first()
        existing_email = User.query.filter_by(email=form.email.data).first()

        if existing_user:
            flash('이미 존재하는 아이디입니다.')
        elif existing_email:
            flash('이미 존재하는 이메일입니다.')
        else:
            user = User(
                userid=form.userid.data,
                username=form.username.data,
                password=generate_password_hash(form.password1.data),
                email=form.email.data,
                phone=form.phone.data.replace('-', ''),
                birth=datetime.strptime(form.birth.data, "%Y-%m-%d").date(),
                Terms_of_Service=form.Terms_of_Service.data,
                Privacy_Policy=form.Privacy_Policy.data,
                receive_emails=form.receive_emails.data
            )
            db.session.add(user)
            db.session.commit()
            return redirect(url_for('auth.login'))

    return render_template('auth/signup.html', form=form)
```

HTML

```
{{ form.csrf_token }}
{% include 'form_errors.html' %}

{{ form.userid.label(class="mt-5") }}
{{ form.userid(class="form-control mb-3") }}

{{ form.password1.label }}
{{ form.password1(class="form-control mb-3") }}

{{ form.password2.label }}
{{ form.password2(class="form-control mb-3") }}

{{ form.username.label }}
{{ form.username(class="form-control mb-3") }}

{{ form.email.label }}
{{ form.email(class="form-control mb-3") }}

{{ form.phone.label }}
{{ form.phone(class="form-control mb-3") }}

{{ form.birth.label }}
{{ form.birth(class="form-control mb-4") }}
```

데이터베이스: user

	id	username	password	email	userid	s_of_Se	vac_Pol	eve_en	status	_admi	phone	birth	created_at	admin_role
1	김동환	script:...	test01@test.com	iensud184	1	1	0	normal	0	111111111111	2001-12-01		NULL	NULL
2	김동환	script:...	test02@test.com	test1000	1	1	0	normal	0	111111111111	2001-12-01		NULL	NULL
3	김동환	script:...	test123@test.com	test123	1	1	0	normal	0	111111111111	1111-11-11	2026-04-21 06:52:02.419076	NULL	NULL
4	슈퍼관리자	script:...	admin@test.com	admin	1	1	0	normal	1	010999999999	2000-01-01	2026-04-22 01:32:12.744540	super	super
5	운영관리자	script:...	manager@test.com	manager	1	1	0	normal	1	010999999999	2000-01-01	2026-04-22 01:32:12.958924	manager	manager
6	김동환	script:...	test1234@test.com	test1234	1	1	0	normal	0	010111111111	2001-11-11	2026-04-27 04:20:28.254652	none	none
7	김동환	script:...	test12345@test.com	test12345	1	1	0	normal	0	010111111111	1111-11-11	2026-04-27 05:11:27.579034	none	none
8	김동환	script:...	test888@test.com	test888	1	1	0	normal	0	111111111111	1111-11-11	2026-04-27 05:17:27.046781	none	none

DataBase

CODE (로그인)

로그인 페이지 화면구현

로그인

아이디와 비밀번호를 입력하세요.

아이디

비밀번호

아이디 찾기

비밀번호 찾기

로그인

회원가입

아이디를 입력하세요

이름을 입력하세요

이메일을 입력하세요

전화번호 (- 없이)

비밀번호 재설정

Python

```
# 데코레이션 함수
def login_required(view):
    @functools.wraps(view)
    def wrapped_view(*args, **kwargs):

        # 로그인 안 된 경우
        if g.user is None:
            flash('로그인이 필요한 서비스입니다.')
            _next = request.url if request.method == 'GET' else ''
            return redirect(url_for('auth.login', next=_next))

        # 휴면회원 강제 이동
        if g.user.status == 'sleep':
            allow_pages = [
                'auth.sleep_member',
                'auth.wake_member',
                'auth.logout'
            ]

            if request.endpoint not in allow_pages:
                return redirect(url_for('auth.sleep_member'))

        return view(*args, **kwargs)

    return wrapped_view
```

```
elif form.validate_on_submit():
    user = User.query.filter_by(userid=form.userid.data).first()

    if not user:
        flash('존재하지 않는 아이디입니다.')
    elif not check_password_hash(user.password, form.password.data):
        flash('비밀번호가 올바르지 않습니다.')
    else:
        session.clear()
        session['user_id'] = user.id

        if user.status == 'sleep':
            return redirect(url_for('auth.sleep_member'))

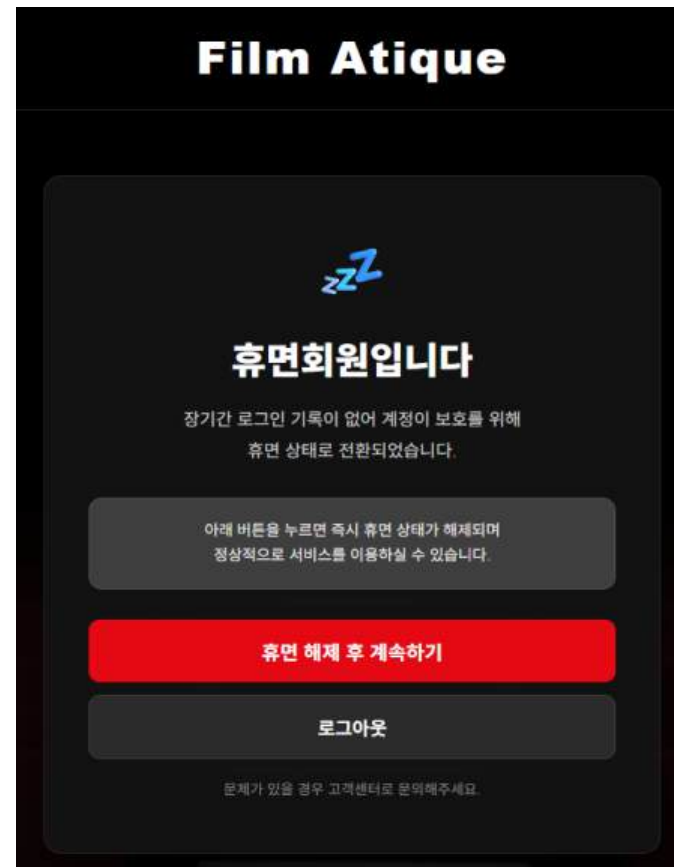
        if user.is_admin:
            return redirect(url_for('auth.admin'))

        return redirect(url_for('main.index'))

#라우팅 함수보다 먼저 실행
@bp.before_app_request
def load_logged_in_user():
    user_id = session.get('user_id')
    if user_id is None:
        g.user = None
    else:
        g.user = db.session.get(User, user_id)
```


CODE (휴면 회원)

휴면 회원 로그인시 화면구현



Python

```
def login_required(view):
    @functools.wraps(view)
    def wrapped_view(*args, **kwargs):
        # 로그인 안 된 경우
        if g.user is None:
            flash('로그인이 필요한 서비스입니다.')
            _next = request.url if request.method == 'GET' else ''
            return redirect(url_for('auth.login', next=_next))

        # 휴면회원 강제 이동
        if g.user.status == 'sleep':
            allow_pages = [
                'auth.sleep_member',
                'auth.wake_member',
                'auth.logout'
            ]

            if request.endpoint not in allow_pages:
                return redirect(url_for('auth.sleep_member'))

        return view(*args, **kwargs)
    return wrapped_view

@bp.route('/sleep-member')
@login_required
def sleep_member():
    if g.user.status != 'sleep':
        return redirect(url_for('main.index'))

    return render_template('auth/sleep_member.html')

@bp.route('/wake-member')
@login_required
def wake_member():
    if g.user.status == 'sleep':
        g.user.status = 'normal'
        db.session.commit()

    flash('휴면회원이 해제되었습니다.')
    return redirect(url_for('main.index'))
```

HTML

```
<header>
  <div class="headermain">
    <div id="logo">
      <a href="{{ url_for('main.index') }}">Film Atique</a>
    </div>
  </div>
</header>

<main>
  <div class="sleep-wrap">
    <div class="sleep-card">
      <div class="icon">zzZ</div>

      <h1>휴면회원입니다</h1>

      <p class="desc">
        장기간 로그인 기록이 없어 계정이 보호를 위해<br>
        휴면 상태로 전환되었습니다.
      </p>

      <div class="info-box">
        아래 버튼을 누르면 즉시 휴면 상태가 해제되며<br>
        정상적으로 서비스를 이용하실 수 있습니다.
      </div>

      <a href="{{ url_for('auth.wake_member') }}" class="btn-release">
        휴면 해제 후 계속하기
      </a>

      <a href="{{ url_for('auth.logout') }}" class="btn-logout">
        로그아웃
      </a>

      <p class="sub-text">
        문제가 있을 경우 고객센터로 문의해주세요.
      </p>
    </div>
  </div>
</main>
```

CODE (아이디, 비밀번호 찾기)

로그인 페이지 화면구현

로그인
아이디와 비밀번호를 입력하세요.

아이디
비밀번호

아이디 찾기 비밀번호 찾기

로그인
회원가입

아이디를 입력하세요
이름을 입력하세요
이메일을 입력하세요
전화번호 (- 없이)

비밀번호 재설정

HTML

```
{% if reset_mode %}
<h5 class="my-3 pb-1">비밀번호 재설정</h5>
<h6 class="pb-3">새로운 비밀번호를 입력하세요.</h6>

<form method="post" action="{{ url_for('auth.login') }}">
  <input type="hidden" name="action" value="reset">
  <input type="hidden" name="user_id" value="{{ reset_user_id }}">
  <input type="password"
    name="new_password"
    placeholder="새 비밀번호"
    class="form-control mb-3">

  <button type="submit" class="btn btn-danger w-100">
    변경하기
  </button>
</form>

<div class="text-center mt-3">
  <a href="{{ url_for('auth.login') }}">로그인으로 돌아가기</a>
</div>
{% endif %}
```

python

```
@bp.route('/find-id', methods=['POST'])
def find_id():
    email = request.form.get('email')
    user = User.query.filter_by(email=email).first()

    if user:
        flash(f'아이디는 {user.userid} 입니다.')
    else:
        flash('해당 이메일이 존재하지 않습니다.')

    return redirect(url_for('auth.login'))
```

Python

```
@bp.route('/find-password', methods=['POST'])
def find_password():
    userid = request.form.get('userid')
    username = request.form.get('username')
    email = request.form.get('email')
    phone = request.form.get('phone')

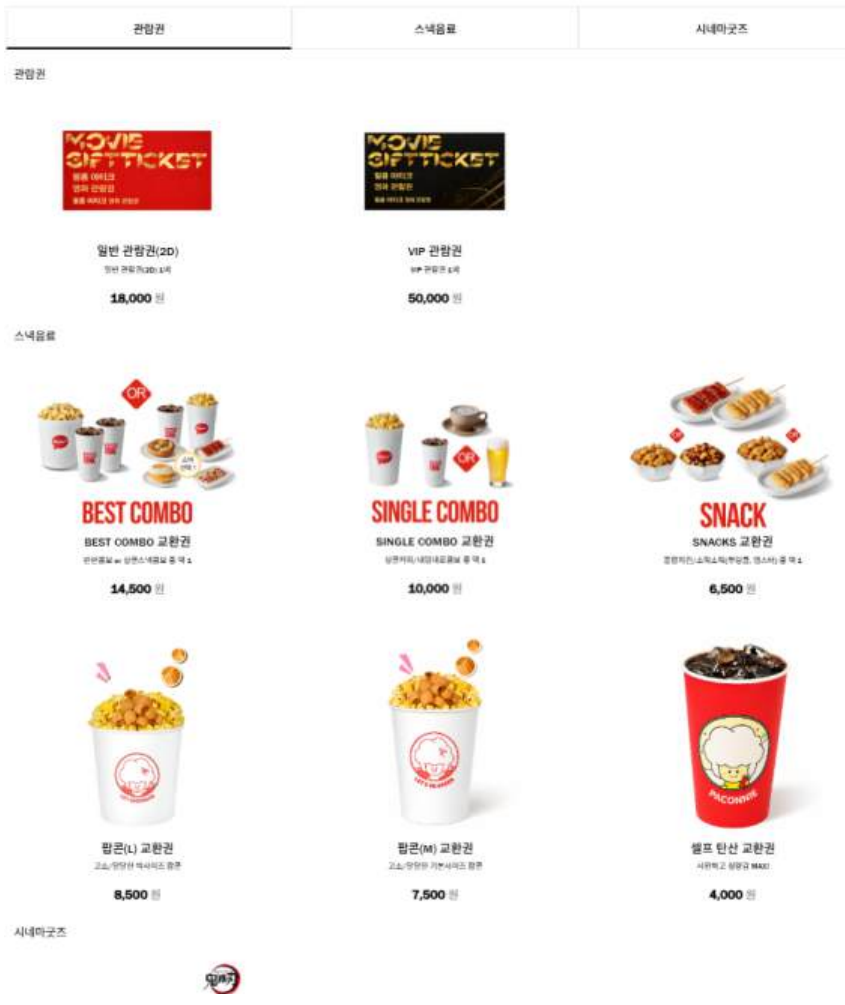
    # 전화번호 하이픈 제거
    if phone:
        phone = phone.replace('-', '')

    user = User.query.filter_by(
        userid=userid,
        username=username,
        email=email,
        phone=phone
    ).first()

    if user:
        return render_template(
            'auth/login.html',
            form=UserLoginForm(),
            reset_mode=True,
            reset_user_id=user.id
        )
    else:
        flash('입력한 정보와 일치하는 계정을 찾을 수 없습니다.')
        return redirect(url_for('auth.login'))
```


CODE (상품 스토어)

상품 스토어 페이지 화면구현



DataBase

테이블(T): product										
	id	Productname	Producttype	Productprice	stock	Productdescription	Productimage_url	Productlimit	Productdate	status
1	1	일반 관람권 (2D)	티켓	18000	1000	일반 관람권 (2D) 1매	/static/img/redticket.png	10	온라인관람권 3 개월	normal
2	2	VIP 관람권	티켓	50000	1000	VIP 관람권 1매	/static/img/blackticket.png	2	온라인관람권 3 개월	normal
3	3	BEST COMBO 교환권	스낵	14500	1000	반반콤보 or 싱글스낵콤보 중 택 1	/static/img/BEST COMBO 교환권.png	10	스위트샵 상품권 3 개월	normal
4	4	SINGLE COMBO 교환권	스낵	10000	1000	싱글커피/내맘대로콤보 중 택 1	/static/img/SINGLE COMBO 교환권.png	10	스위트샵 상품권 3 개월	normal
5	5	SNACKS 교환권	스낵	6500	1000	팝콘치킨/소먹소먹 (뿌링클, 맵스터) 중 택 1	/static/img/SNACKS 교환권.png	10	스위트샵 상품권 3 개월	normal
6	6	팝콘 (L) 교환권	스낵	8500	1000	고소/달달한 빅사이즈 팝콘	/static/img/popcorn_L.png	10	스위트샵 상품권 3 개월	normal
7	7	팝콘 (M) 교환권	스낵	7500	1000	고소/달달한 기본사이즈 팝콘	/static/img/popcorn_M.png	10	스위트샵 상품권 3 개월	normal
8	8	셀프 탄산 교환권	스낵	4000	1000	시원하고 청량감 MAX!	/static/img/drink_L.png	10	스위트샵 상품권 3 개월	normal
9	9	귀멸의 칼날 랜덤 홀로그램 캔벤티 교환권	굿즈	9900	1000	홀로그램 캔벤티	/static/img/...	3	스위트샵 상품권 3 개월	normal
10	10	귀멸의 칼날 랜덤 포토카라 캔벤티 교환권	굿즈	9900	1000	캐릭터 캔벤티	/static/img/...	3	스위트샵 상품권 3 개월	normal
11	11	귀멸의 칼날 랜덤 그라픽 캔벤티 교환권	굿즈	9900	1000	그라픽 캔벤티	/static/img/...	3	스위트샵 상품권 3 개월	normal
12	12	진격의 거인 아크릴 예거 키홀더 교환권	굿즈	22500	1000	진격의 거인 아크릴 키홀더	/static/img/jin_eren.jpg	3	스위트샵 상품권 3 개월	normal
13	13	진격의 거인 아크릴 미카사 키홀더 교환권	굿즈	22500	1000	진격의 거인 VIVID 아크릴 키홀더	/static/img/jin_mikasa.jpg	3	스위트샵 상품권 3 개월	normal
14	14	진격의 거인 아크릴 리바이 키홀더 교환권	굿즈	22500	1000	진격의 거인 VIVID 아크릴 키홀더	/static/img/jin_levi.jpg	3	스위트샵 상품권 3 개월	normal
15	15	체인소 맨 덴지 아크릴스탠드 교환권	굿즈	30000	1000	체인소 맨 아크릴스탠드	/static/	{% for product in products if product.Producttype == '티켓' %}		
16	16	체인소 맨 마키마 아크릴스탠드 교환권	굿즈	30000	1000	체인소 맨 아크릴스탠드	/static/	<a href="{{ url_for('store.product', id=product.id) }}"		
17	17	체인소 맨 레제 아크릴스탠드 교환권	굿즈	30000	1000	체인소 맨 아크릴스탠드	/static/	class="product-card {% if product.status == 'soldout' %}soldout-item{% endif %}">		

```
// ease-out 스크롤 함수
function smoothScrollTo(targetY, duration = 200) {
  const startY = window.scrollY;
  const distance = targetY - startY;
  const startTime = performance.now();

  function easeOutCubic(t) {
    return 1 - Math.pow(1 - t, 3);
  }

  function scrollStep(currentTime) {
    const time = Math.min(1, (currentTime - startTime) / duration);
    const eased = easeOutCubic(time);

    window.scrollTo(0, startY + distance * eased);

    if (time < 1) {
      requestAnimationFrame(scrollStep);
    }
  }

  requestAnimationFrame(scrollStep);
}
```

JavaScript

```
<a href="{{ url_for('store.product', id=product.id) }}"
class="product-card {% if product.status == 'soldout' %}soldout-item{% endif %}">

  {% if product.status == 'soldout' %}
  <div class="soldout-badge">품절</div>
  {% endif %}

  <div class="nor_img">
    
  </div>

  <div class="nor_cont">
    <h4>{{ product.Productname }}</h4>
    <p>{{ product.Productdescription }}</p>
  </div>

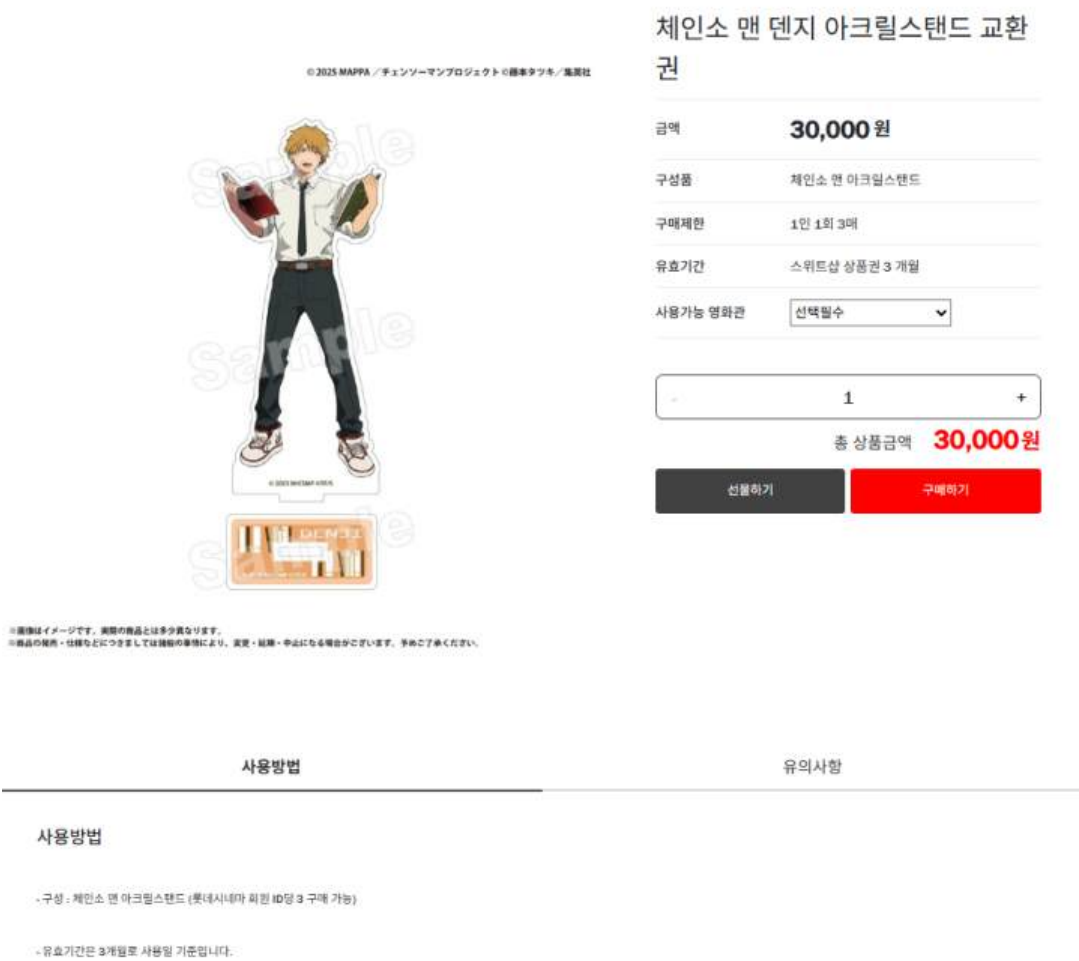
  <div class="nor_price">
    <span>{{ "{:,}".format(product.Productprice) }}</span>
    <em>원</em>
  </div>
</a>

{% endfor %}
```

HTML

CODE (상품 상세)

상품 상세페이지 화면구현



JavaScript

```
// 수량 증가
plusBtn.addEventListener("click", function () {
    let count = parseInt(numBox.textContent);

    if (count < maxCount) {
        count++;
        updateUI(count);
    }
});

// UI 전체 업데이트
function updateUI(count) {
    numBox.textContent = count;

    // 가격 업데이트
    priceBox.innerHTML = (price * count).toLocaleString() + "<em>원</em>";

    // 버튼 비활성화 처리
    plusBtn.disabled = (count >= maxCount);
    minusBtn.disabled = (count <= minCount);
}

<script>
    const productprice = {{ product.Productprice }};
</script>
<script>const Productlimit = {{ product.Productlimit }};</script>

// 수량 감소
minusBtn.addEventListener("click", function () {
    let count = parseInt(numBox.textContent);

    if (count > minCount) {
        count--;
        updateUI(count);
    }
});
```

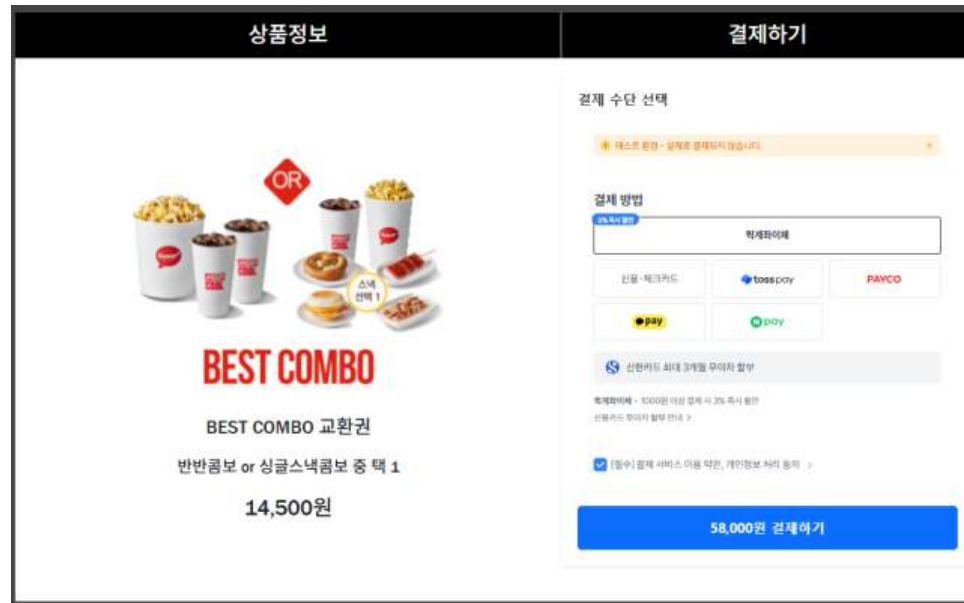
Python

```
@bp.route("/main")
def store_main():
    products = Product.query.all()
    return render_template("store.html", products=products)

@bp.route("/product/<int:id>")
def product(id):
    product = Product.query.get_or_404(id)
    return render_template("product.html", product=product)
```


CODE (상품 결제)

상품 결제 페이지 화면구현



JavaScript

```
<script src="https://js.tosspayments.com/v1/payment-widget"></script>
<script>
  const clientKey = "test_gck_docs_Ovk5rk1EwkEbP0W43n07x1zm";
  const customerKey = "USER_{{ session.get('user_id') }}";

  const paymentWidget = PaymentWidget(clientKey, PaymentWidget.ANONYMOUS);

  const orderId = "{{ order.order_code }}";
  const amount = "{{ order.total_price }}";

  // 결제 UI
  paymentWidget.renderPaymentMethods("#payment-method", { value: amount });
  paymentWidget.renderAgreement("#agreement");

  const paymentButton = document.getElementById("payment-button");

  paymentButton.addEventListener("click", function () {

    paymentWidget.requestPayment({
      orderId: orderId, // DB 주문 ID 사용
      orderName: "{{ order.product.Productname }}",
      amount: amount, // 총 금액 사용
      successUrl: window.location.origin + "/store/pay/success?order_id=" + orderId,
      failUrl: window.location.origin + "/store/pay/fail?order_id=" + orderId
    });

  });
</script>
```

Python

```
@bp.route('/pay')
def store_pay():
    order_id = request.args.get('order_id')

    order = Order.query.filter_by(order_code=order_id).first_or_404()
    payment = Payment.query.filter_by(order_id=order.id).first()

    if not payment:
        payment = Payment(
            order_id=order.id,
            order_code=order.order_code,
            amount=order.total_price,
            status="READY"
        )
        db.session.add(payment)
        db.session.commit()

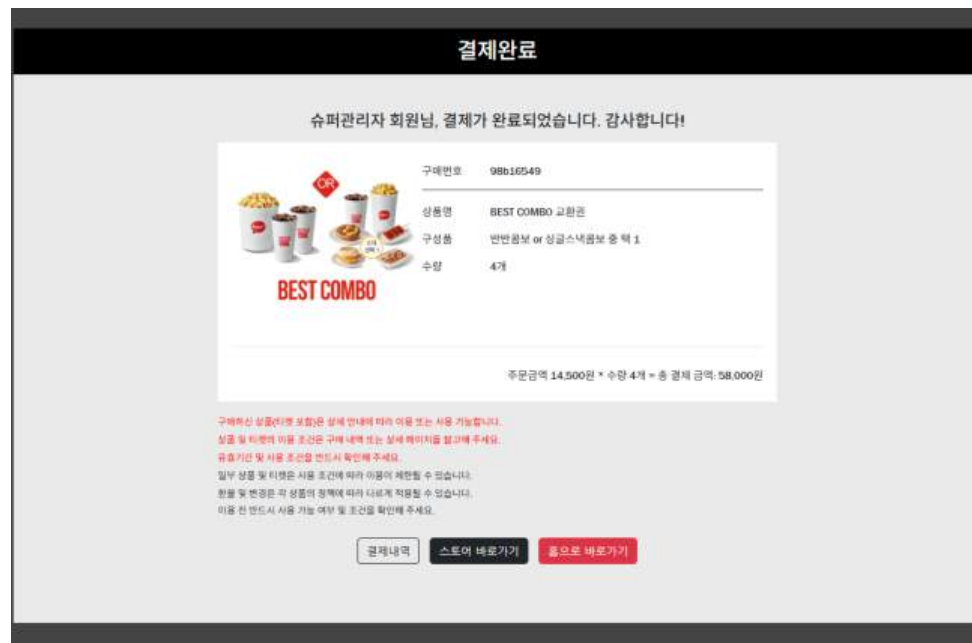
    return render_template('store_pay.html', order=order)
```

HTML

```
<div class="menu_name_main">
  <div class="product_img">
    
  </div>
  <div class="product_name">
    <h3>{{ order.product.Productname }}</h3>
  </div>
  <div class="product_detail">
    <h4>{{ order.product.Productdescription }}</h4>
  </div>
  <div class="product_price">
    <h2>{{ "{:,}".format(order.product.Productprice) }}원</h2>
  </div>
</div>
```


CODE (상품 결제 성공)

상품 결제 성공 페이지 화면구현



Python

```
@bp.route('/pay/success')
def pay_success():
    order_code = request.args.get("order_id")

    # 토스에서 오는 값
    payment_key = request.args.get("paymentKey")
    method = request.args.get("method")

    order = Order.query.filter_by(order_code=order_code).first_or_404()
    payment = Payment.query.filter_by(order_id=order.id).first()

    # Payment 업데이트
    payment.payment_key = payment_key
    payment.method = method
    payment.status = "SUCCESS"
    payment.approved_at = datetime.now(timezone.utc)

    # Order도 성공 처리
    order.status = "SUCCESS"

    db.session.commit()

    return render_template('storepay_success.html', order=order)
```

```
@bp.route('/pay/fail')
def pay_fail():
    order_code = request.args.get("order_id")

    order = Order.query.filter_by(order_code=order_code).first()

    if order:
        payment = Payment.query.filter_by(order_id=order.id).first()

        if payment:
            payment.status = "FAIL"

        order.status = "FAIL"
        db.session.commit()

    return f"결제 실패 (order_id={order_code})"
```

DataBase

테이블(T): payment									
	id	order_code	method	amount	requested_at	approved_at	order_id	payment_key	status
...	필터		필터	필터	필터	필터	필터	필터	필터
1	1	order_2_1_fb956e3a	NULL	100000	2026-04-21 03:29:30	2026-04-21 03:50:02.299692	26	tgen_20260421123030XKMf5	SUCCESS
2	2	order_4_1_59c07e49	NULL	70000	2026-04-21 03:50:32	2026-04-21 04:11:31.460340	27	tgen_20260421125033WihH9	SUCCESS
3	3	order_1_2_2a81cc74	NULL	72000	2026-04-21 05:37:02	2026-04-21 05:37:56.975151	28	tgen_20260421143705XU0q0	SUCCESS

CODE (영화리스트)

영화리스트 페이지 화면구현

현재 상영작



프로젝트 헤일메리



셸터



리 크로닌의 미이라



크라임 101



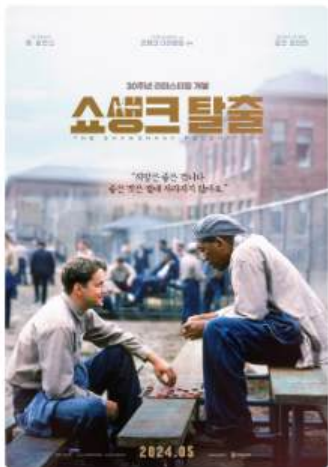
고스트: 더 레전드



극장판 귀멸의 칼날: 무한성편



스크림 7



쇼생크 탈출



HTML

```
{% block content %}
<div id="movie_list">
  <h3>현재 상영작</h3>
  <div class="movie">
    {% for movie in movies %}
    <div class="box">
      

      <h4>{{ movie.title }}</h4>

      <div class="txtbox">
        <a href="{{ url_for('film.booking', movie_id=movie.id) }}">예매하기</a>
        <a href="/film/movie/{{ movie.tmdb_id }}">상세보기</a>
      </div>
    </div>
    {% endfor %}
  </div>
{% endblock %}
```

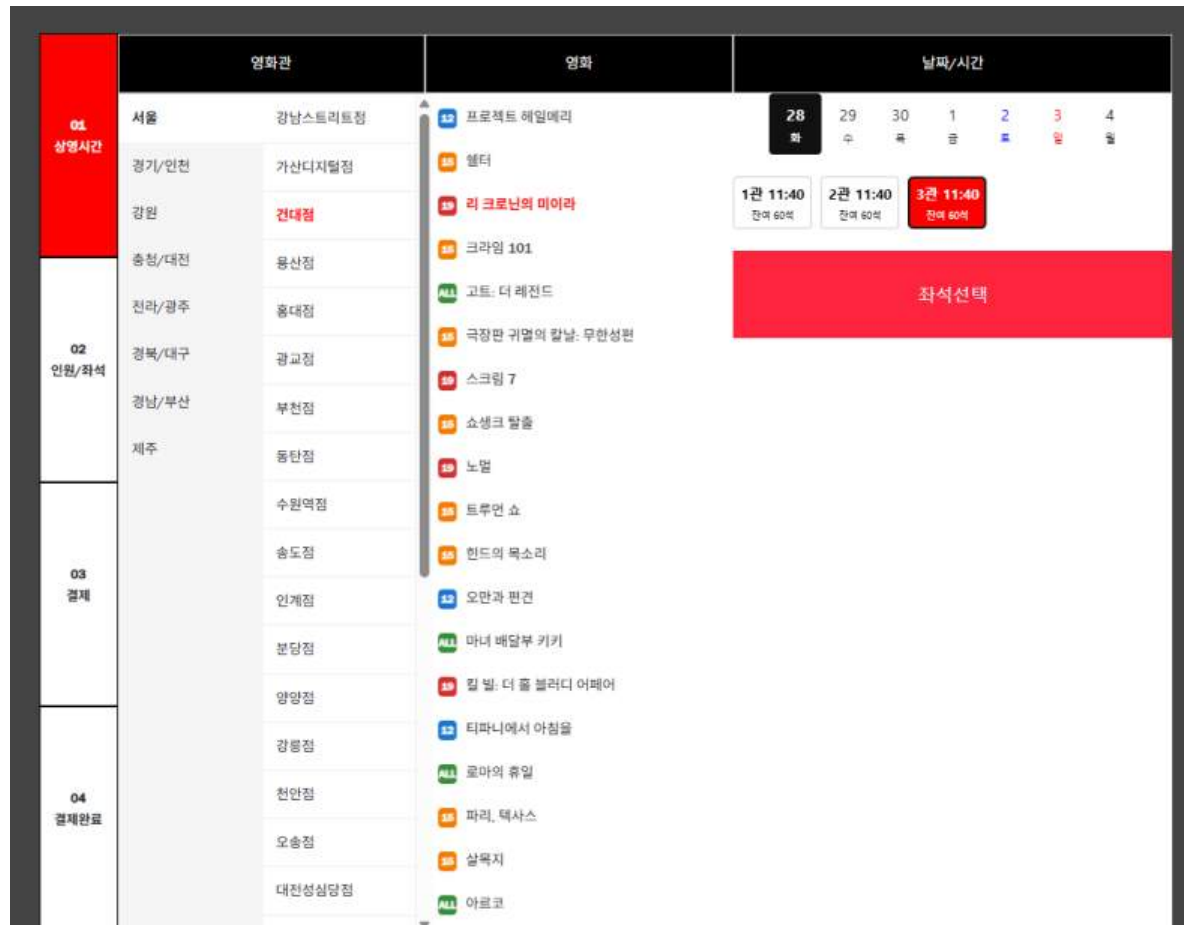
id	tmdb_id	title	overview	poster_path	release_date	vote_average	certification	genres	
1	687163	프로젝트 헤일메리	눈을 떠보니 아득한 우주의 한가운데에서 ...	/qqGpVVZk2KD11AvocgTU426nh1H.jpg	2026-03-18	8.2	12	SF, 모험	라이언 고슬링, 산
2	1290821	셸터	등대에 홀로 숨어있던 한 남자. 그 앞에 홀로...	/tS3U21VZE504z8inIZXG1HNqDfE.jpg	2026-04-15	6.772	15	액션, 범죄,	제이슨 스타뎀, 보
3	1304313	리 크로닌의 미이라	한 기자의 어린 딸이 흔적도 없이 사라져 ...	/zX1wtzXP80yc3X0VokdxvmsFQfw.jpg	2026-04-22	6.849	19	공포,	잭 레이너, 라이마
4	1171145	크라임 101	101번 국도를 따라 흔적도 없이 사라지는 ...	/77gpcowG00ORQY9x33NeBIFajm1.jpg	2026-04-08	6.979	15	범죄,	크리스 험스워스,
5	1297842	고스트: 더 레전드	큰 꿈을 품은 작은 영소 윌은 세계에서 가장 ...	/otP94vckeMXAgQxzhcRkZSeSmYv.jpg	2026-04-17	7.908	ALL	애니메이션, 코미디,	케일링 맥러플린,
6	1311031	극장판 귀멸의 칼날: 무한성편	혈귀로 변해버린 여동생 네츠코를 인간으로 ...	/m6Dhc6hDCcLSKI8mOQNemZAedFI.jpg	2025-08-22	7.681	15	애니메이션, 액션, 판타지	하나에 나츠키, 사
7	1159559	스크림 7	시드니 프레스콧이 새롭게 정착한 조용한 ...	/gqgBqxyr8tGQJCFrRMAzFA7Cm1.jpg	2026-04-01	6.03	19	공포, , 범죄	니브 캠벨, 코트니
8	278	쇼생크 탈출	속망받는 은행 간부 앤디 듀프레인은 아내와 ...	/qV9BQZd1M8foEzDz0Ag5hGWE5qM.jpg	1995-01-28	8.718	15	드라마, 범죄	짐 로빈스, 모건
9	1242332	노멀	눈 덮인, 조용하고 평범한 마을 노멀. 보안관 ...	/A7TN0R78eenIf7hED3x96POTJ0X.jpg	2026-04-17	7.2	19	액션, 범죄,	밥 오웬커크, 헨리
10	37165	트루먼 쇼	트루먼 버뱅크는 작고 조용한 섬마을에 사는 ...	/AsUv4pFF1UQuOLKE7nYUXgmGCHf.jpg	1998-10-24	8.2	15	코미디, 드라마	짐 캐리, 로라 리
11	1480382	한티의 목소리	2024년 1월 29일, 가자지구에 대피 명령이 ...	/rHIVG1EBHnAskQsg1x1XSMJy7td.jpg	2026-04-15	8.044	15	드라마	رافي إيداد رجب
12	4348	오만과 편견	베넷가의 다섯 자매 중 둘째인 엘리자베스는 ...	/rgKfV60FgUBIFR6cN1CukRnEMga.jpg	2006-03-24	8.049	12	드라마, 로맨스	키이라 나이틀리,
13	16859	마녀 배달부 키키	마녀 얼마와 인간 아빠 사이에서 태어난 소녀 ...	/yEbyzR1eLDoZVIJv6auSr9yW5ZG.jpg	2007-11-22	7.834	ALL	애니메이션, , 판타지	타카야마 미나미,
14	414419	릴 빌: 더 폴 블러디 에피어	23년 만에 공개되는 완전판, 275분 동안 ...	/je5CQCvXoAhWAw16MoCa18IFWC.jpg	2026-04-01	8.1	19	액션, 범죄,	루마 서먼, 루시
15	164	티파니에서 아침을	1940년대 초 뉴욕, 검은 선글라스에 화려한 ...	/8U6b8j5wT8FZ4k57sV0vEwpKkY.jpg	1962-10-01	7.623	12	코미디, 로맨스, 드라마	오드리 헵번, Geo
16	804	로마의 휴일	유럽 순방 중 로마를 방문한 공주 '앤 ...	/dYLiHwYkoxC2I6bJU1k4ZHnMRko.jpg	1955-09-30	7.891	ALL	로맨스, 코미디, 드라마	오드리 헵번, 그레
17	655	파리, 텍사스	미국과 멕시코의 국경 사이, 붉은 모자를 쓴 ...	/3urLYGDMKJave5Yus1hhtc8nuMU.jpg	1987-08-29	8.096	15	드라마	해리 딘 스타턴, L
18	1461254	살목지	기묘한 소문이 끊이지 않는 저수지 살목지의 ...	/uVJyW1SPCWftRUXM61xTqOzFV6.jpg	2026-04-08	0.0	15	공포	김혜운, 이종원, 김
19	804370	마르코	2075년, 자연이 너무 위험해져 아이들은 돔 ...	/452f14x08vqzafp1E19fzK7Zs4D.jpg	2026-03-11	7.483	ALL	애니메이션, 모험, 판타지, , SF	Margot Ringard
20	1100099	위 리브 인 타임	본인의 레스토랑을 오픈을 준비하며 새로운 ...	/18HFN8Xx9Cen4CkSWHk94t6gHM.jpg	2026-04-08	7.224	15	로맨스, 드라마	앤드류 가필드, 플

DataBase

영화 데이터 DB에 저장 후 호출

CODE (예매)

영화예매 페이지 화면구현



```
<div class="theater_list flex-column">
  <ul class="list-unstyled">
    <% for theater in theaters %>
      <li class="{% if loop.first %}selected{% endif %}" data-region="{ {{ theater.region.name }}"
        data-theater-id="{ {{ theater.id }}">
        {{ theater.name }}
      </li>
    <% endfor %>
  </ul>
</div>

<!-- 영화 -->
<div class="select_group movie">
  <ol id="movieList">
    <% for movie in movies %>
      <li class="movie_title {% if movie.id == movie_id %}selected{% endif %}" data-movie-id="{ {{ movie.id }}">
        <!-- 관람등급 -->
        <span class="movie_age">
          <% if movie.certification == '19' %>age-19
          <% elif movie.certification == '15' %>age-15
          <% elif movie.certification == '12' %>age-12
          <% else %>age-all
          <% endif %>
          <% if movie.certification == '정보없음' %>
            ALL
          <% else %>
            {{ movie.certification }}
          <% endif %>
        </span>
        <!-- 제목 -->
        {{ movie.title }}
      </li>
    <% endfor %>
  </ol>
</div>
```

HTML

```
@bp.route("/booking/<int:movie_id", methods=['GET'])
@login_required
def booking(movie_id):
    movies = Movie.query.all()

    # 선택된 영화
    movie = Movie.query.get(movie_id)

    if not movie:
        abort(404)

    theaters = Theater.query.all()

    # 날짜 리스트 (종목 제거)
    dates = db.session.query(
        func.date(Schedule.start_time)
    ).filter(
        Schedule.movie_id == movie_id
    ).distinct().all()

    # 기본 날짜 하나 선택 (첫 번째)
    selected_date = dates[0][0] if dates else None

    # 해당 날짜 시간들
    schedules = []
    if selected_date:
        schedules = Schedule.query.filter(
            Schedule.movie_id == movie_id,
            func.date(Schedule.start_time) == selected_date
        ).all()

    return render_template(
        'booking.html',
        movie_id=movie_id,
        movies=movies,
        movie=movie,
        theaters=theaters,
        dates=dates,
        schedules=schedules,
        selected_date=selected_date
    )
```

Python

```
function loadSchedules() {
  if (!selected.movie || !selected.date) return;

  fetch(`/film/api/schedules?movie_id=${selected.movie}&date=${selected.date}&theater_id=${selected.theater}`)
    .then(res => res.json())
    .then(data => renderTimes(data));
}
```

Java Script

```
@bp.route("/api/schedules")
def get_schedules():
    movie_id = request.args.get('movie_id', type=int)
    date_str = request.args.get('date')
    theater_id = request.args.get('theater_id', type=int)

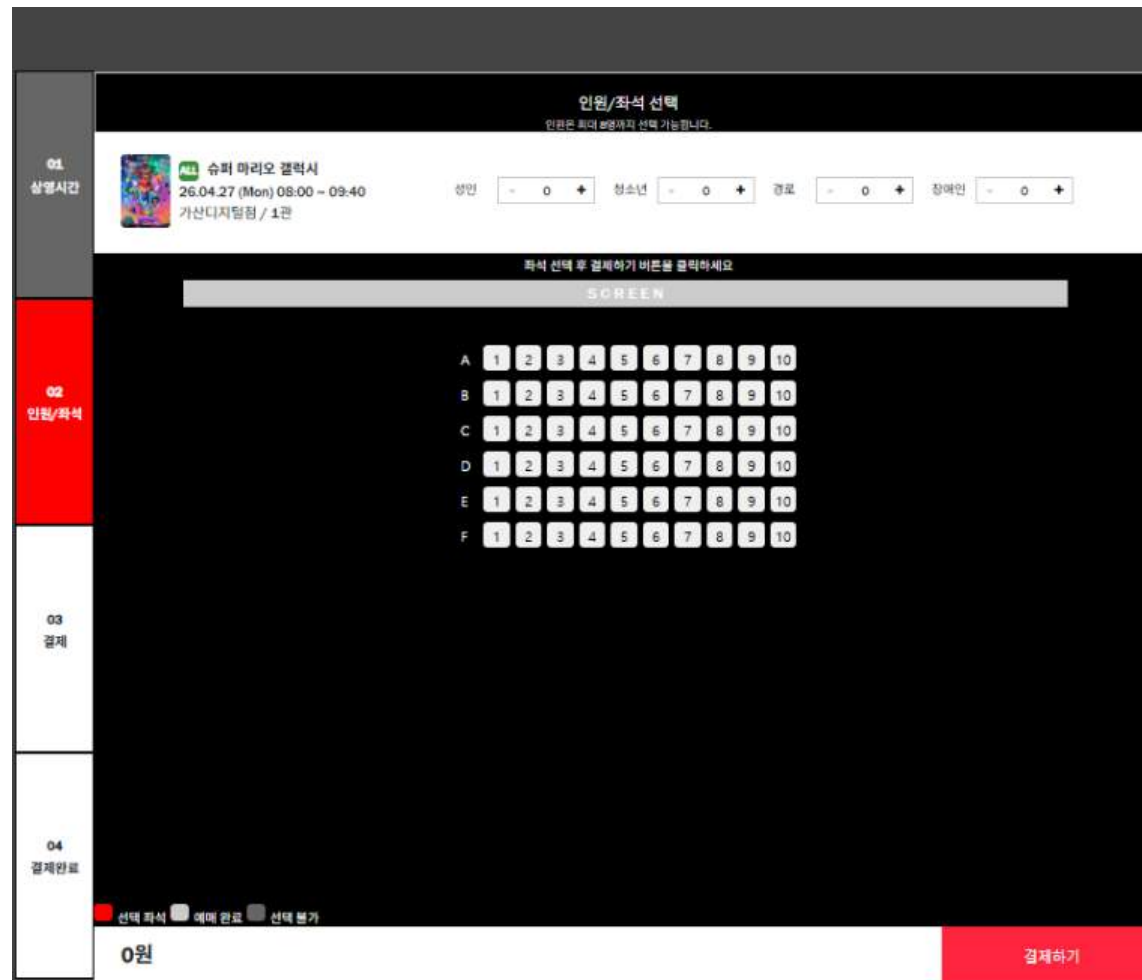
    date = datetime.strptime(date_str, "%Y-%m-%d")

    schedules = Schedule.query.join(Screen).join(Theater).filter(
        Schedule.movie_id == movie_id,
        func.date(Schedule.start_time) == date.date(),
        Theater.id == theater_id
    ).all()

    result = []
```


CODE (좌석선택)

좌석선택 페이지 화면구현



```
// 좌석 생성
const rows = ["A", "B", "C", "D", "E", "F"];
const cols = 10;

rows.forEach(row => {

  const rowDiv = document.createElement("div");
  rowDiv.classList.add("seat_row");

  const label = document.createElement("span");
  label.classList.add("row_label");
  label.innerText = row;

  rowDiv.appendChild(label);

  for (let i = 1; i <= cols; i++) {
    const seat = document.createElement("button");

    seat.classList.add("seat");
    seat.innerText = i;
    seat.dataset.seat = `${row}${i}`;

    rowDiv.appendChild(seat);

    seatArea.appendChild(rowDiv);
  }
});

const scheduleId = new URLSearchParams(location.search).get("schedule_id");

fetch(`/film/api/reserved_seats?schedule_id=${scheduleId}`)
  .then(res => res.json())
  .then(reservedSeats => {

    reservedSeats.forEach(seatCode => {
      const seatEl = document.querySelector(`[data-seat="${seatCode}"]`);

      if (seatEl) {
        seatEl.classList.add("disabled");
        seatEl.style.background = "#ccc";
        seatEl.style.cursor = "not-allowed";
      }
    });
  });

// 가격표
const prices = [20000, 12000, 7000, 5000];

// 총 인원
function getTotalPeople() {
  let total = 0;
  numBoxes.forEach(box => {
    total += parseInt(box.textContent);
  });
  return total;
}

// 총 금액
function calculatePrice() {
  let totalPrice = 0;

  numBoxes.forEach((box, index) => {
    totalPrice += parseInt(box.textContent) * prices[index];
  });

  return totalPrice;
}

// 좌석 선택
seatArea.addEventListener("click", (e) => {

  const seat = e.target.closest(".seat");
  if (!seat) return;

  if (seat.classList.contains("disabled")) return;

  const seatId = seat.dataset.seat;
  const totalPeople = getTotalPeople();

  // 인원 선택 안 했을 때
  if (totalPeople === 0) {
    alert("인원을 먼저 선택하세요!");
    return;
  }

  // 선택 해제
  if (seat.classList.contains("selected")) {
    seat.classList.remove("selected");

    const index = selectedSeats.indexOf(seatId);
    if (index > -1) selectedSeats.splice(index, 1);
  } else {
    // 인원 초과 방지
    if (selectedSeats.length >= totalPeople) {
      alert(`좌석은 ${totalPeople}개까지만 선택 가능합니다.`);
      return;
    }

    seat.classList.add("selected");
    selectedSeats.push(seatId);
  }

  console.log("좌석:", selectedSeats);
});
```

Java Script

```
<div class="persontotal">
  <div class="selectmovie">
    <!-- 포스터 -->
    <div class="poster">
      
    </div>

    <!-- 정보 -->
    <div class="info">
      <!-- 등급 + 제목 -->
      <div class="title">
        <span class="age">
          {% if movie.certification == '19' %}age-19
          {% elif movie.certification == '15' %}age-15
          {% elif movie.certification == '12' %}age-12
          {% else %}age-all
          {% endif %}
        </span>
        {{ movie.certification if movie.certification != '정보없음' else 'ALL' }}
      </div>
      <strong>{{ movie.title }}</strong>
    </div>

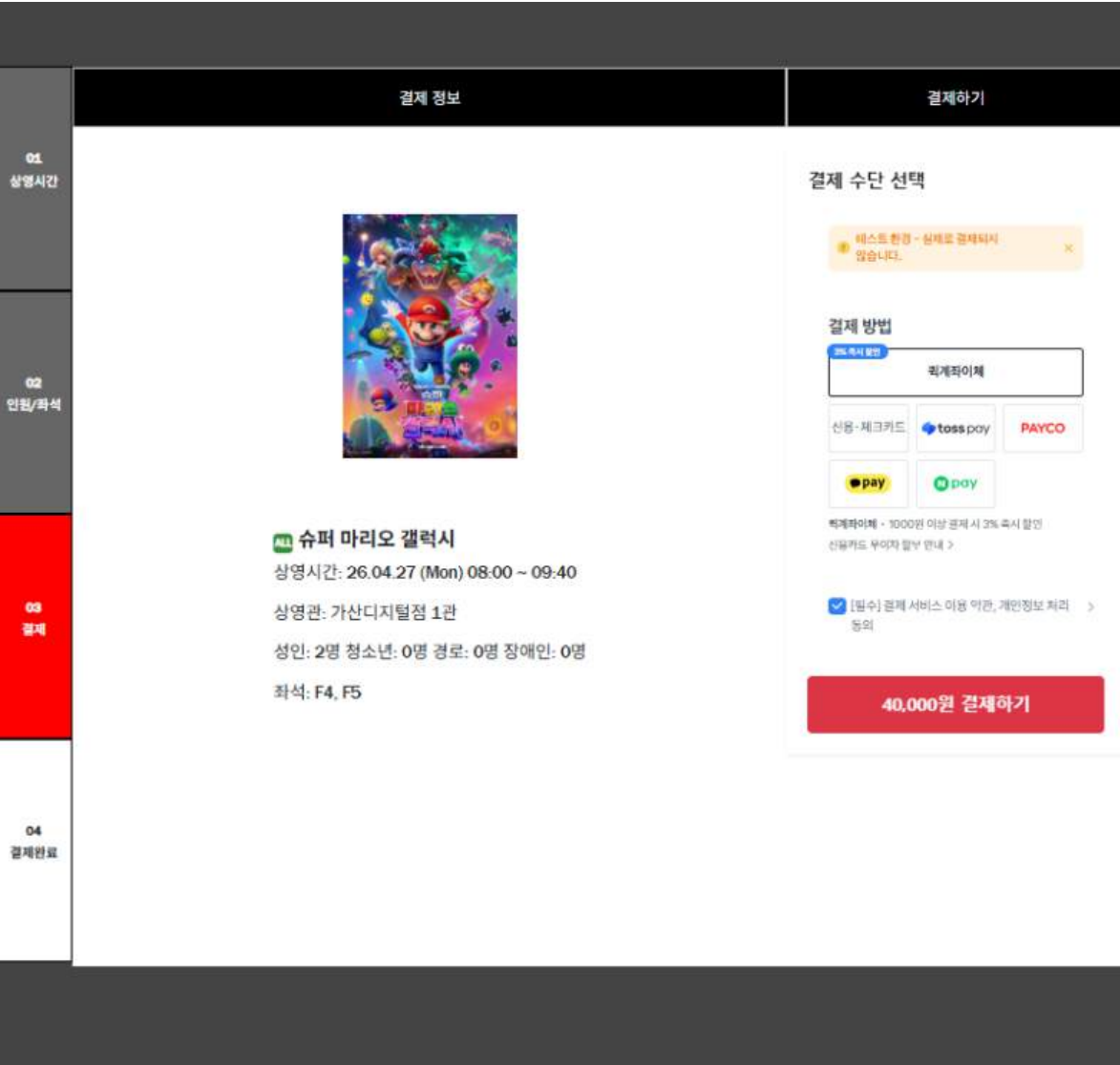
    <!-- 날짜 + 시간 -->
    <div class="datetime">
      {{ schedule.start_time.strftime("%y.%m.%d (%a)") }}
      {{ schedule.start_time.strftime("%H:%M") }} ~
      {{ schedule.end_time.strftime("%H:%M") }}
    </div>

    <!-- 극장 정보 -->
    <div class="location">
      {{ theater.name }} / {{ screen.name }}
    </div>
  </div>
</div>
```

HTML

CODE (예매결제)

예매결제 페이지 화면구현



```
<script src="https://js.tosspayments.com/v1/payment-widget"></script>
<script>
  const userId = "{{ session.get('user_id') }}";
  const orderCode = "{{ order.order_code }}";
  const totalAmount = "{{ order.total_price }}";
  const movieTitle = "{{ movie.title }}";
  const seatCount = {{ seats | length }};

</script>
<script>
  const clientKey = "test_gck_docs_Ovk5rk1EwkEbP0W43n87x1zm";
  const customerKey = "USER_{{ session.get('user_id') }}";

  // 로그인 기반으로 변경
  const paymentWidget = PaymentWidget(clientKey, customerKey);

  const orderId = "{{ order.order_code }}";
  const amount = {{ order.total_price }};

  // 결제 UI 렌더링
  paymentWidget.renderPaymentMethods("payment-method", { value: amount });
  paymentWidget.renderAgreement("#agreement");

  const paymentButton = document.getElementById("payment-button");

  // 버튼 텍스트 설정
  paymentButton.innerText = amount.toLocaleString() + "원 결제하기";

  paymentButton.addEventListener("click", async function () {

    // 중복 클릭 방지
    paymentButton.disabled = true;

    try {
      // 서버에 예매 데이터 먼저 저장
      const res = await fetch("/film/movie/payment", {
        method: "POST",
        headers: {
          "Content-Type": "application/json",
          "X-Requested-With": "XMLHttpRequest"
        },
        body: JSON.stringify({
          schedule_id: {{ schedule.id }},
          seats: {{ seats | tojson }},
          people: {{ people | tojson }},
          total_price: {{ total_price }}
        })
      });

      if (!res.ok) {
        const data = await res.json();

        if (data.message === "adult_required") {
          window.location.href = data.redirect;
          return;
        }
      }

      alert("결제 실패");
      paymentButton.disabled = false;
      return;
    }
  });
}
```

```
const result = await res.json();

if (!result.success) {
  alert("예매 데이터 저장 실패");
  paymentButton.disabled = false;
  return;
}

// 저장 성공 후 결제 요청
paymentWidget.requestPayment({
  orderId: orderId,
  orderName: "{{ movie.title }} ({{ seats | length }}석)",
  amount: amount,
  successUrl: window.location.origin + "/film/payment/success?order_id=" + orderId,
  failUrl: window.location.origin + "/film/payment/fail?order_id=" + orderId
});

} catch (err) {
  console.error(err);
  alert("서버 오류 발생");
  paymentButton.disabled = false;
}
```

Java Script(TOSS 결제 API)

```
<div class="payment_sys">
  <div class="card border-0 shadow-sm p-4 mt-4" style="min-height: 500px;">
    <h5 class="fw-bold mb-3">결제 수단 선택</h5>

    <div id="payment-method"></div>

    <div id="agreement"></div>
    {# 결제 버튼에 총 결제 금액 표시 #}
    <button id="payment-button" class="btn btn-danger w-100 py-3 fw-bold mt-3 fs-5"></button>
  </div>
</div>
```

HTML

```
@bp.route('/movie/payment', methods=['GET', 'POST'])
@login_required
def movie_payment():

    if request.method == 'POST':
        data = request.get_json()
        schedule_id = data.get("schedule_id") if data else None
        session['booking_data'] = data

        return jsonify({"success": True})

    schedule_id = request.args.get('schedule_id', type=int)
    schedule = Schedule.query.get_or_404(schedule_id)

    movie = schedule.movie
    screen = schedule.screen
    theater = screen.theater

    payment_data = session.get('booking_data')
    if not payment_data:
        return redirect(url_for("film.person_seat", schedule_id=schedule.id))

    seats = payment_data.get('seats')
    people = payment_data.get('people')
    total_price = payment_data.get('total_price')

    num_people = sum(people.values())

    order_code = str(uuid.uuid4())

    order = Order(
        order_code=order_code,
        user_id=g.user.id,
        total_price=total_price,
        status="READY",

        seats_json=json.dumps(seats),
        people_json=json.dumps(people),
        schedule_id=schedule.id
    )

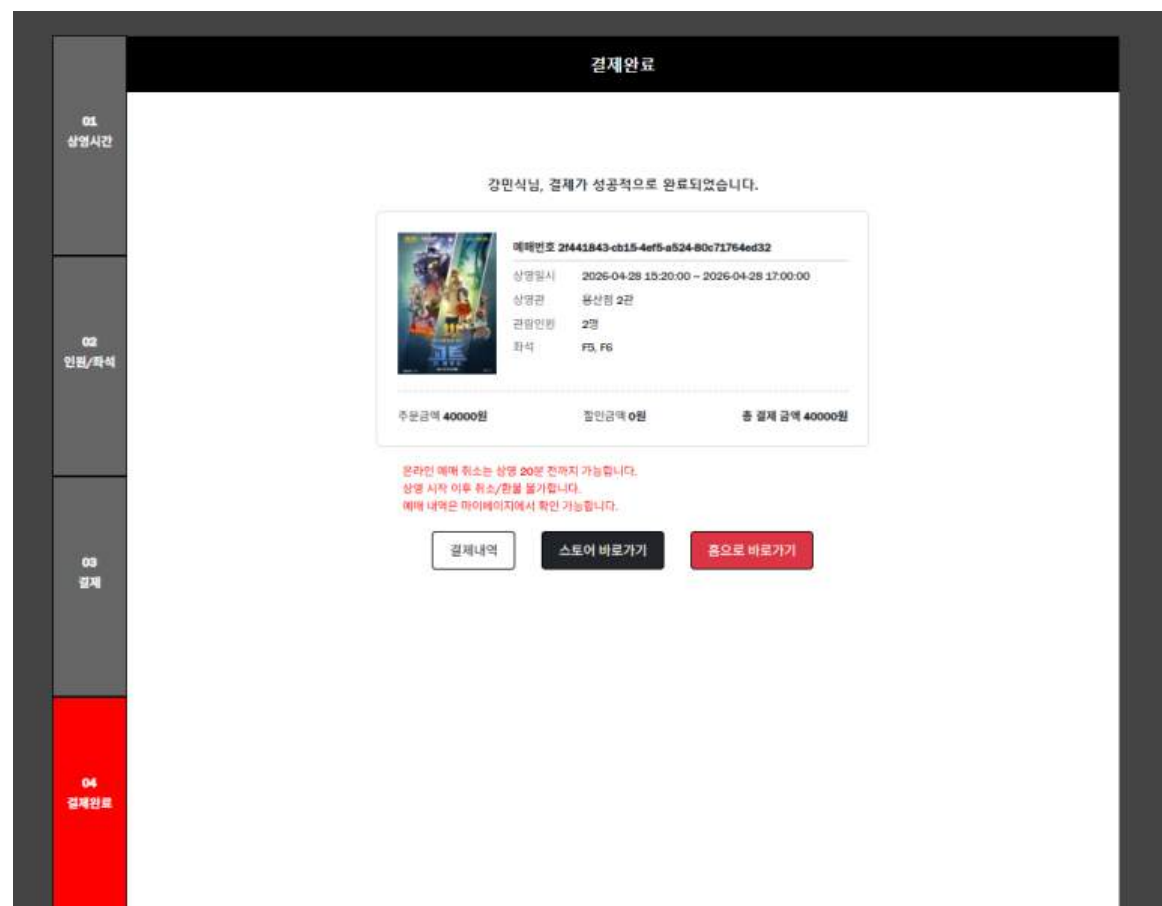
    db.session.add(order)
    db.session.commit()

    return render_template(
        'movie_payment.html',
        schedule=schedule,
        movie=movie,
        screen=screen,
        theater=theater,
        seats=seats,
        people=people,
        num_people=num_people,
        total_price=total_price,
        order=order,
    )
```

Python

CODE (예매결제완료)

예매 결제완료 페이지 화면구현



```
@bp.route('/payment/success')
@login_required
def payment_success():

    order_id = request.args.get("order_id")

    if not order_id:
        return redirect(url_for("main.index"))

    order = Order.query.filter_by(order_code=order_id).first()

    if not order:
        return redirect(url_for("main.index"))

    seats = json.loads(order.seats_json)
    people = json.loads(order.people_json)
    schedule = Schedule.query.get(order.schedule_id)

    if order.status == "SUCCESS":
        return render_template(
            "payment_success.html",
            movie=schedule.movie,
            schedule=schedule,
            theater=schedule.screen.theater,
            screen=schedule.screen,
            seats=seats,
            total_price=order.total_price,
            people=people,
            booking_code=order.order_code
        )

    user_id = session.get("user_id")

    booking_code = order.order_code
    order.status = "SUCCESS"

    reserved_seats = []

    for seat_code in seats:
        row = seat_code[0]
        col = int(seat_code[1:])

        seat = Seat.query.filter_by(
            screen_id=schedule.screen_id,
            row=row,
            col=col
        ).first()

        existing = Reservation.query.filter_by(
            schedule_id=schedule.id,
            seat_id=seat.id
        ).first()

        if existing:
            continue

        reservation = Reservation(
            user_id=user_id,
            schedule_id=schedule.id,
            seat_id=seat.id,
            created_at=datetime.now()
        )

        db.session.add(reservation)
        reserved_seats.append(seat_code)

    db.session.commit()

    return render_template(
```

Python

```
<div class="title">결제완료</div>

<div class="complete_msg">
    <p><strong>{{ g.user.username }}님, 결제가 성공적으로 완료되었습니다.</strong></p>
</div>

<!-- 티켓 -->
<div class="ticket">

    <div class="ticket_top">
        

        <div class="info">
            <p class="booking">예매번호 <strong>{{ booking_code }}</strong></p>
            <p><span>상영일시</span> {{ schedule.start_time }} ~ {{ schedule.end_time }}</p>
            <p><span>상영관</span> {{ theater.name }} {{ screen.name }}</p>
            <p><span>관람인원</span> {{ people.adult }}명</p>
            <p><span>좌석</span> {{ seats | join(', ') }}</p>
        </div>
    </div>

    <div class="divider"></div>

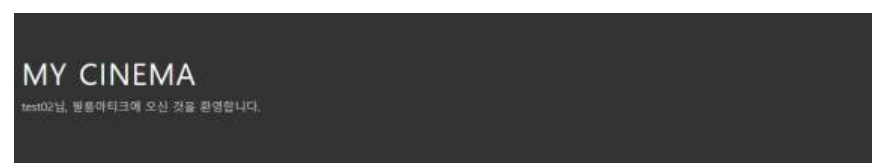
    <div class="ticket_bottom">
        <div>주문금액 <strong>{{ total_price }}원</strong></div>
        <div>환입금액 <strong>0원</strong></div>
        <div class="total">총 결제 금액 <strong>{{ total_price }}원</strong></div>
    </div>

</div>
```

HTML

CODE (마이페이지)

마이페이지 화면구현



Python

```
@bp.route('/mypage', methods=['GET'])
def mypage():
    user_id = session.get('user_id')

    if not user_id:
        return redirect(url_for('auth.login'))

    user = User.query.get(user_id)

    from collections import defaultdict

    # 현재 예매 내역
    active = Reservation.query.filter_by(
        user_id=user.id,
        status='RESERVED'
    ).all()

    grouped = defaultdict(list)

    for r in active:
        grouped[r.schedule_id].append(r)

    reservation_list = []

    for schedule_id, items in grouped.items():
        schedule = items[0].schedule
        seats = [f"{i.seat.row}-{i.seat.col}" for i in items]

        reservation_list.append({
            "schedule_id": schedule_id,
            "movie": schedule.movie.title,
            "theater": schedule.screen.theater.name,
            "datetime": schedule.start_time,
            "seats": ", ".join(seats)
        })
```

```
# 취소 내역 (통합)
cancels = []

# 예매 취소
canceled_reservations = Reservation.query.filter_by(
    user_id=user.id,
    status='CANCEL'
).all()

cancel_group = defaultdict(list)

for r in canceled_reservations:
    cancel_group[r.schedule_id].append(r)

for schedule_id, items in cancel_group.items():
    schedule = items[0].schedule

    cancels.append({
        "type": "reservation",
        "movie_title": schedule.movie.title,
        "cancel_date": items[0].created_at.strftime("%Y-%m-%d %H:%M")
    })

# 결제 취소
cancel_payments = Payment.query\
    .join(Order)\
    .filter(
        Order.user_id == user.id,
        Payment.status == "CANCELLED"
    )\
    .all()

for p in cancel_payments:
    cancels.append({
        "type": "payment",
        "movie_title": p.order.product_name,
        "cancel_date": p.approved_at.strftime("%Y-%m-%d %H:%M") if p.approved_at else "-"
    })

# 결제 완료 내역
payment = Payment.query\
    .join(Order)\
    .filter(
        Order.user_id == user.id,
        Payment.status == "SUCCESS"
    )\
    .all()

orders = Order.query.filter_by(user_id=user.id).all()
```

```
return render_template(
    'mypage.html',
    user=user,
    reservations=reservation_list,
    cancels=cancels,
    payment=payment,
    orders=orders
)
```

```
@bp.route('/pay/cancel/<int:order_id>', methods=['POST'])
@login_required
def cancel_payment(order_id):

    order = Order.query.get_or_404(order_id)
    payment = Payment.query.filter_by(order_id=order.id).first()

    if not payment:
        return jsonify({"message": "결제 정보 없음"}), 400

    if payment.status != "SUCCESS":
        return jsonify({"message": "취소 불가 상태"}), 400

    # 70일 제한 (선택)
    if payment.approved_at:
        now = datetime.now(timezone.utc)
        approved_at = payment.approved_at.replace(tzinfo=timezone.utc) \
            if payment.approved_at.tzinfo is None else payment.approved_at

        if now > approved_at + timedelta(days=70):
            return jsonify({"message": "취소 기간 초과"}), 400

    # 토스 없이 DB만 처리
    payment.status = "CANCELLED"
    order.status = "CANCELLED"

    db.session.commit()

    return jsonify({"message": "취소 완료"})
```

```
@bp.route('/reservation/cancel/<int:schedule_id>', methods=['POST'])
def cancel_reservation(schedule_id):
    user_id = session.get('user_id')

    if not user_id:
        return redirect(url_for('auth.login'))

    reservations = Reservation.query.filter_by(
        user_id=user_id,
        schedule_id=schedule_id,
        status='RESERVED'
    ).all()

    for r in reservations:
        r.status = 'CANCEL'

    db.session.commit()
    flash('예매가 취소되었습니다.')

    return redirect(url_for('film.mypage'))
```


CODE (공지사항)

공지사항 화면구현

영화관	제목	등록일
강남스트리트점	강남스트리트점 2026년 12월 유무일 안내	2026-12
가산디지털점	가산디지털점 2026년 12월 유무일 안내	2026-12
건대점	건대점 2026년 12월 유무일 안내	2026-12
용산점	용산점 2026년 12월 유무일 안내	2026-12
롯데점	롯데점 2026년 12월 유무일 안내	2026-12
광교점	광교점 2026년 12월 유무일 안내	2026-12
부천점	부천점 2026년 12월 유무일 안내	2026-12
동탄점	동탄점 2026년 12월 유무일 안내	2026-12
수원역점	수원역점 2026년 12월 유무일 안내	2026-12
송도점	송도점 2026년 12월 유무일 안내	2026-12
인계점	인계점 2026년 12월 유무일 안내	2026-12
분당점	분당점 2026년 12월 유무일 안내	2026-12
양양점	양양점 2026년 12월 유무일 안내	2026-12
강릉점	강릉점 2026년 12월 유무일 안내	2026-12
천안점	천안점 2026년 12월 유무일 안내	2026-12

이전 1 2 3 4 5 ... 47 48 다음

공지사항 디테일 화면구현

영화관	제목	등록일
강남스트리트점	강남스트리트점 2026년 12월 유무일 안내	2026년 12월 01일 AM 12:00

안녕하세요, 필름아티크입니다.
먼저 필름아티크 강남스트리트점을 이용해주시는 고객님께 항상 깊은 감사 드립니다.

필름아티크 강남스트리트점은 67스퀘어 건물 전체 의무 휴업일에 따라
12월 9일(화)과 12월 29일(화) 휴관합니다.
그 외 모든 요일은 정상 영업 예정이니 이용에 참고 부탁드립니다.

※ 의무 휴업 기준 : 제 2018-243호 대형마트 및 중대규모점포 의무휴업일 지정 및 영업시간 제한 고시

감사합니다.

이전 : 서귀포점 2026년 11월 유무일 안내
다음 : 가산디지털점 2026년 12월 유무일 안내

목록

seed_questions.py로 데이터를 저장하였음

Python

```
# notice_list
@app.route("/notice/notice_list/")
def notice_list():
    page = request.args.get('page', type=int, default=1)

    notice_list=Notice.query.order_by(Notice.created_date.desc())

    notice_list = notice_list.paginate(page=page, per_page=15) # 페이지당 보여야 할 게시물

    return render_template("cs/notice/notice_list.html", notice_list=notice_list)

# notice_detail
@app.route("/notice/detail/<int:notice_id>")
def notice_detail(notice_id):

    notice = Notice.query.get_or_404(notice_id)

    # 이전 글 (현재 글보다 id 작은 것 중 가장 큰 값)
    prev_notice = Notice.query.filter(
        Notice.id < notice_id
    ).order_by(Notice.id.desc()).first()

    # 다음 글 (현재 글보다 id 큰 것 중 가장 작은 값)
    next_notice = Notice.query.filter(
        Notice.id > notice_id
    ).order_by(Notice.id.asc()).first()

    return render_template(
        "cs/notice/notice_detail.html",
        notice=notice,
        prev_notice=prev_notice,
        next_notice=next_notice
    )
```

Model

```
# notice
class Notice(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    theater = db.Column(db.String(100), nullable=False)
    title = db.Column(db.String(200), nullable=False)
    content = db.Column(db.Text(), nullable=False)
    created_date = db.Column(db.DateTime, nullable=False)
```

CODE (리뷰 1:1문의)

문의(1:1문의) 화면구현

review_form으로 이동 <= [문의하기](#)

문의유형	영향권	제목	등록일	처리상태
건의	강남스트리트점	추가질문 등록하기	2026-04-28	답변대기
질문	강남스트리트점	ss	2026-04-24	답변대기
질문	강남스트리트점	마니 마니 r3123123	2026-04-24	답변대기
질문	강남스트리트점	추가질문 등록하기 L	2026-04-23	답변완료
문의	강남스트리트점	123	2026-04-23	답변대기
suggestion	1	영양제	2026-04-23	답변대기
suggestion	1	오예전	2026-04-22	답변대기
etc	12	8888	2026-04-22	답변대기
suggestion	1	오예전	2026-04-22	답변대기
suggestion	1	추가질문 등록하기	2026-04-22	답변대기

이전 1 2 다음

문의작성(1:1문의) 화면구현

문의작성

문의유형 ☐ 문의 ☐ 건의 ☐ 칭찬 ☐ 불만 ☐ 기타

영향권 강남스트리트점

제목

내용

첨부 파일 파일 선택 선택된 파일 없음

[저장](#) [취소](#)

HTML

```

52 <div class="mb-3 row border-bottom pb-2">
53 <label class="col-sm-2 col-form-label">
54 {{ form.cs_ask.label.text }}
55 </label>
56
57 <div class="col-sm-5">
58 {% for subfield in form.cs_ask %}
59 <div class="form-check form-check-inline">
60 {{ subfield(class="form-check-input") }}
61 <label class="form-check-label" for="{{ subfield.id }}">
62 {{ subfield.label.text }}
63 </label>
64 </div>
65 {% endfor %}
66 </div>
67
68 {#영향권 장소 cs_place#}
69 <div class="mb-4 row border-bottom pb-3">
70 <div class="col-sm-2">
71 {{ form.cs_place.label(class="form-label") }}
72 </div>
73 <div class="col-sm-10">
74 {{ form.cs_place(class="form-select") }}
75 </div>
76 </div>

```

=> 라디오 체크박스

=> 장소선택 박스

Python

```

237 @login_required
238 def review_create():
239     form = ReviewForm()
240     if request.method == 'POST' and form.validate_on_submit():
241         image_files = form.image.data
242         image_paths = []
243
244         # 저장 경로 : 오늘 날짜로 폴더 생성
245         today = datetime.now().strftime('%Y%m%d')
246         upload_folder = os.path.join(current_app.root_path, 'static/photo', today)
247         os.makedirs(upload_folder, exist_ok=True)
248
249         if image_files:
250             for image_file in image_files:
251                 # 파일이 실패한 비어있지 않은지 확인
252                 if image_file and image_file.filename != '':
253                     filename = secure_filename(image_file.filename)
254                     file_path = os.path.join(upload_folder, filename)
255                     image_file.save(file_path)
256
257                 # DB를 저장 경로 리스트에 추가
258                 image_paths.append(f'photo/{today}/{filename}')
259             joined_image_paths = ",".join(image_paths) if image_paths else None
260
261         review = Review(
262             cs_ask=form.cs_ask.data,
263             cs_place=form.cs_place.data,
264             subject=form.subject.data,
265             content=form.content.data,
266             created_date=datetime.now(),
267             image_path=joined_image_paths,
268             user=g.user
269         )
270         db.session.add(review)
271         db.session.commit()
272
273         return redirect(url_for('cs.review_list'))
274     return render_template('template_name_or_list: 'cs/review/review_form.html', form=form)
275

```

Python

```

:22 # 1:1 문의 목록
:23 @bp.route("/review/") @ login_required
:24 @login_required
:25 def review_list():
:26     page = request.args.get('page', type=int, default=1)
:27     review_list=Review.query.order_by(Review.created_date.desc())
:28
:29     review_list = review_list.paginate(page=page, per_page=10)
:30
:31
:32     return render_template( template_name_or_list: "cs/review/review.html", review_list=review_list)
:33

```

파일 이미지 첨부 <=

```

99 <div class="col-sm-2">
100 <label for="formFileMultiple" class="form-label">첨부<br>
101 파일</label>
102 </div>
103 <div class="col-sm-10">
104 <input class="form-control" type="file" name="image" id="formFileMultiple" multiple>
105 </div>
106 </div>
107 <div class="btn-group custom wt-3">
108 <button type="submit" class="btn btn-danger btn-fixed" style="width: 100px;">
109 저장
110 </button>
111 <button type="button" class="btn btn-secondary btn-fixed"
112 onclick="location.href='{{ url_for('cs.review_list') }}'" style="width: 100px;">
113 취소
114 </button>
115 </div>
116 </form>
117 </table>
118 </div>
119 </div>
120 </div>
121 </div>
122 </div>
123 </div>
124 </div>
125 </div>
126 </div>
127 </div>
128 </div>
129 </div>
130 </div>
131 </div>
132 </div>
133 </div>
134 </div>
135 </div>
136 </div>
137 </div>
138 </div>
139 </div>
140 </div>
141 </div>
142 </div>
143 </div>
144 </div>
145 </div>
146 </div>
147 </div>
148 </div>
149 </div>
150 </div>
151 </div>
152 </div>
153 </div>
154 </div>
155 </div>
156 </div>
157 </div>
158 </div>
159 </div>
160 </div>
161 </div>
162 </div>
163 </div>
164 </div>
165 </div>
166 </div>
167 </div>
168 </div>
169 </div>
170 </div>
171 </div>
172 </div>
173 </div>
174 </div>
175 </div>
176 </div>
177 </div>
178 </div>
179 </div>
180 </div>
181 </div>
182 </div>
183 </div>
184 </div>
185 </div>
186 </div>
187 </div>
188 </div>
189 </div>
190 </div>
191 </div>
192 </div>
193 </div>
194 </div>
195 </div>
196 </div>
197 </div>
198 </div>
199 </div>
200 </div>
201 </div>
202 </div>
203 </div>
204 </div>
205 </div>
206 </div>
207 </div>
208 </div>
209 </div>
210 </div>
211 </div>
212 </div>
213 </div>
214 </div>
215 </div>
216 </div>
217 </div>
218 </div>
219 </div>
220 </div>
221 </div>
222 </div>
223 </div>
224 </div>
225 </div>
226 </div>
227 </div>
228 </div>
229 </div>
230 </div>
231 </div>
232 </div>
233 </div>
234 </div>
235 </div>
236 </div>
237 </div>
238 </div>
239 </div>
240 </div>
241 </div>
242 </div>
243 </div>
244 </div>
245 </div>
246 </div>
247 </div>
248 </div>
249 </div>
250 </div>
251 </div>
252 </div>
253 </div>
254 </div>
255 </div>
256 </div>
257 </div>
258 </div>
259 </div>
260 </div>
261 </div>
262 </div>
263 </div>
264 </div>
265 </div>
266 </div>
267 </div>
268 </div>
269 </div>
270 </div>
271 </div>
272 </div>
273 </div>
274 </div>
275 </div>
276 </div>
277 </div>
278 </div>
279 </div>
280 </div>
281 </div>
282 </div>
283 </div>
284 </div>
285 </div>
286 </div>
287 </div>
288 </div>
289 </div>
290 </div>
291 </div>
292 </div>
293 </div>
294 </div>
295 </div>
296 </div>
297 </div>
298 </div>
299 </div>
300 </div>
301 </div>
302 </div>
303 </div>
304 </div>
305 </div>
306 </div>
307 </div>
308 </div>
309 </div>
310 </div>
311 </div>
312 </div>
313 </div>
314 </div>
315 </div>
316 </div>
317 </div>
318 </div>
319 </div>
320 </div>
321 </div>
322 </div>
323 </div>
324 </div>
325 </div>
326 </div>
327 </div>
328 </div>
329 </div>
330 </div>
331 </div>
332 </div>
333 </div>
334 </div>
335 </div>
336 </div>
337 </div>
338 </div>
339 </div>
340 </div>
341 </div>
342 </div>
343 </div>
344 </div>
345 </div>
346 </div>
347 </div>
348 </div>
349 </div>
350 </div>
351 </div>
352 </div>
353 </div>
354 </div>
355 </div>
356 </div>
357 </div>
358 </div>
359 </div>
360 </div>
361 </div>
362 </div>
363 </div>
364 </div>
365 </div>
366 </div>
367 </div>
368 </div>
369 </div>
370 </div>
371 </div>
372 </div>
373 </div>
374 </div>
375 </div>
376 </div>
377 </div>
378 </div>
379 </div>
380 </div>
381 </div>
382 </div>
383 </div>
384 </div>
385 </div>
386 </div>
387 </div>
388 </div>
389 </div>
390 </div>
391 </div>
392 </div>
393 </div>
394 </div>
395 </div>
396 </div>
397 </div>
398 </div>
399 </div>
400 </div>
401 </div>
402 </div>
403 </div>
404 </div>
405 </div>
406 </div>
407 </div>
408 </div>
409 </div>
410 </div>
411 </div>
412 </div>
413 </div>
414 </div>
415 </div>
416 </div>
417 </div>
418 </div>
419 </div>
420 </div>
421 </div>
422 </div>
423 </div>
424 </div>
425 </div>
426 </div>
427 </div>
428 </div>
429 </div>
430 </div>
431 </div>
432 </div>
433 </div>
434 </div>
435 </div>
436 </div>
437 </div>
438 </div>
439 </div>
440 </div>
441 </div>
442 </div>
443 </div>
444 </div>
445 </div>
446 </div>
447 </div>
448 </div>
449 </div>
450 </div>
451 </div>
452 </div>
453 </div>
454 </div>
455 </div>
456 </div>
457 </div>
458 </div>
459 </div>
460 </div>
461 </div>
462 </div>
463 </div>
464 </div>
465 </div>
466 </div>
467 </div>
468 </div>
469 </div>
470 </div>
471 </div>
472 </div>
473 </div>
474 </div>
475 </div>
476 </div>
477 </div>
478 </div>
479 </div>
480 </div>
481 </div>
482 </div>
483 </div>
484 </div>
485 </div>
486 </div>
487 </div>
488 </div>
489 </div>
490 </div>
491 </div>
492 </div>
493 </div>
494 </div>
495 </div>
496 </div>
497 </div>
498 </div>
499 </div>
500 </div>
501 </div>
502 </div>
503 </div>
504 </div>
505 </div>
506 </div>
507 </div>
508 </div>
509 </div>
510 </div>
511 </div>
512 </div>
513 </div>
514 </div>
515 </div>
516 </div>
517 </div>
518 </div>
519 </div>
520 </div>
521 </div>
522 </div>
523 </div>
524 </div>
525 </div>
526 </div>
527 </div>
528 </div>
529 </div>
530 </div>
531 </div>
532 </div>
533 </div>
534 </div>
535 </div>
536 </div>
537 </div>
538 </div>
539 </div>
540 </div>
541 </div>
542 </div>
543 </div>
544 </div>
545 </div>
546 </div>
547 </div>
548 </div>
549 </div>
550 </div>
551 </div>
552 </div>
553 </div>
554 </div>
555 </div>
556 </div>
557 </div>
558 </div>
559 </div>
560 </div>
561 </div>
562 </div>
563 </div>
564 </div>
565 </div>
566 </div>
567 </div>
568 </div>
569 </div>
570 </div>
571 </div>
572 </div>
573 </div>
574 </div>
575 </div>
576 </div>
577 </div>
578 </div>
579 </div>
580 </div>
581 </div>
582 </div>
583 </div>
584 </div>
585 </div>
586 </div>
587 </div>
588 </div>
589 </div>
590 </div>
591 </div>
592 </div>
593 </div>
594 </div>
595 </div>
596 </div>
597 </div>
598 </div>
599 </div>
600 </div>
601 </div>
602 </div>
603 </div>
604 </div>
605 </div>
606 </div>
607 </div>
608 </div>
609 </div>
610 </div>
611 </div>
612 </div>
613 </div>
614 </div>
615 </div>
616 </div>
617 </div>
618 </div>
619 </div>
620 </div>
621 </div>
622 </div>
623 </div>
624 </div>
625 </div>
626 </div>
627 </div>
628 </div>
629 </div>
630 </div>
631 </div>
632 </div>
633 </div>
634 </div>
635 </div>
636 </div>
637 </div>
638 </div>
639 </div>
640 </div>
641 </div>
642 </div>
643 </div>
644 </div>
645 </div>
646 </div>
647 </div>
648 </div>
649 </div>
650 </div>
651 </div>
652 </div>
653 </div>
654 </div>
655 </div>
656 </div>
657 </div>
658 </div>
659 </div>
660 </div>
661 </div>
662 </div>
663 </div>
664 </div>
665 </div>
666 </div>
667 </div>
668 </div>
669 </div>
670 </div>
671 </div>
672 </div>
673 </div>
674 </div>
675 </div>
676 </div>
677 </div>
678 </div>
679 </div>
680 </div>
681 </div>
682 </div>
683 </div>
684 </div>
685 </div>
686 </div>
687 </div>
688 </div>
689 </div>
690 </div>
691 </div>
692 </div>
693 </div>
694 </div>
695 </div>
696 </div>
697 </div>
698 </div>
699 </div>
700 </div>
701 </div>
702 </div>
703 </div>
704 </div>
705 </div>
706 </div>
707 </div>
708 </div>
709 </div>
710 </div>
711 </div>
712 </div>
713 </div>
714 </div>
715 </div>
716 </div>
717 </div>
718 </div>
719 </div>
720 </div>
721 </div>
722 </div>
723 </div>
724 </div>
725 </div>
726 </div>
727 </div>
728 </div>
729 </div>
730 </div>
731 </div>
732 </div>
733 </div>
734 </div>
735 </div>
736 </div>
737 </div>
738 </div>
739 </div>
740 </div>
741 </div>
742 </div>
743 </div>
744 </div>
745 </div>
746 </div>
747 </div>
748 </div>
749 </div>
750 </div>
751 </div>
752 </div>
753 </div>
754 </div>
755 </div>
756 </div>
757 </div>
758 </div>
759 </div>
760 </div>
761 </div>
762 </div>
763 </div>
764 </div>
765 </div>
766 </div>
767 </div>
768 </div>
769 </div>
770 </div>
771 </div>
772 </div>
773 </div>
774 </div>
775 </div>
776 </div>
777 </div>
778 </div>
779 </div>
780 </div>
781 </div>
782 </div>
783 </div>
784 </div>
785 </div>
786 </div>
787 </div>
788 </div>
789 </div>
790 </div>
791 </div>
792 </div>
793 </div>
794 </div>
795 </div>
796 </div>
797 </div>
798 </div>
799 </div>
800 </div>
801 </div>
802 </div>
803 </div>
804 </div>
805 </div>
806 </div>
807 </div>
808 </div>
809 </div>
810 </div>
811 </div>
812 </div>
813 </div>
814 </div>
815 </div>
816 </div>
817 </div>
818 </div>
819 </div>
820 </div>
821 </div>
822 </div>
823 </div>
824 </div>
825 </div>
826 </div>
827 </div>
828 </div>
829 </div>
830 </div>
831 </div>
832 </div>
833 </div>
834 </div>
835 </div>
836 </div>
837 </div>
838 </div>
839 </div>
840 </div>
841 </div>
842 </div>
843 </div>
844 </div>
845 </div>
846 </div>
847 </div>
848 </div>
849 </div>
850 </div>
851 </div>
852 </div>
853 </div>
854 </div>
855 </div>
856 </div>
857 </div>
858 </div>
859 </div>
860 </div>
861 </div>
862 </div>
863 </div>
864 </div>
865 </div>
866 </div>
867 </div>
868 </div>
869 </div>
870 </div>
871 </div>
872 </div>
873 </div>
874 </div>
875 </div>
876 </div>
877 </div>
878 </div>
879 </div>
880 </div>
881 </div>
882 </div>
883 </div>
884 </div>
885 </div>
886 </div>
887 </div>
888 </div>
889 </div>
890 </div>
891 </div>
892 </div>
893 </div>
894 </div>
895 </div>
896 </div>
897 </div>
898 </div>
899 </div>
900 </div>
901 </div>
902 </div>
903 </div>
904 </div>
905 </div>
906 </div>
907 </div>
908 </div>
909 </div>
910 </div>
911 </div>
912 </div>
913 </div>
914 </div>
915 </div>
916 </div>
917 </div>
918 </div>
919 </div>
920 </div>
921 </div>
922 </div>
923 </div>
924 </div>
925 </div>
926 </div>
927 </div>
928 </div>
929 </div>
930 </div>
931 </div>
932 </div>
933 </div>
934 </div>
935 </div>
936 </div>
937 </div>
938 </div>
939 </div>
940 </div>
941 </div>
942 </div>
943 </div>
944 </div>
945 </div>
946 </div>
947 </div>
948 </div>
949 </div>
950 </div>
951 </div>
952 </div>
953 </div>
954 </div>
955 </div>
956 </div>
957 </div>
958 </div>
959 </div>
960 </div>
961 </div>
962 </div>
963 </div>
964 </div>
965 </div>
966 </div>
967 </div>
968 </div>
969 </div>
970 </div>
971 </div>
972 </div>
973 </div>
974 </div>
975 </div>
976 </div>
977 </div>
978 </div>
979 </div>
980 </div>
981 </div>
982 </div>
983 </div>
984 </div>
985 </div>
986 </div>
987 </div>
988 </div>
989 </div>
990 </div>
991 </div>
992 </div>
993 </div>
994 </div>
995 </div>
996 </div>
997 </div>
998 </div>
999 </div>
1000 </div>

```


CODE (리뷰 1:1문의 view)

1:1 이용자 문의 페이지 화면구현

문의유형	영화관	제목	회원	답변상태
1	suggestion	추가질문 등록하기	test01	답변대기

가소소

추가질문 등록하기

가소소

수정 삭제

(관리자 답변)

아직 등록된 답변이 없습니다.

Python

```

304 @bp.route(rule='/review/modify/<int:review_id>', methods=('POST',)) & Y-youngchan +1
305 def review_modify(review_id):
306     review = Review.query.get_or_404(review_id)
307
308     if not (g.user.admin_role in ['super', 'manager'] or not review.answer_review):
309         flash('수정 권한이 없습니다.')
310         return redirect(url_for(endpoint='cs.review_detail', review_id=review.id))
311
312     review.subject = request.form['subject']
313     review.content = request.form['content']
314
315     db.session.commit()
316     flash('수정되었습니다.')
317
318     return redirect(url_for(endpoint='cs.review_detail', review_id=review.id))

```

이용자 리뷰수정

```

323 @bp.route(rule='/review/delete/<int:review_id>', methods=('POST',)) & Y-youngchan +1
324 def review_delete(review_id):
325     review = Review.query.get_or_404(review_id)
326
327     if not (g.user.admin_role in ['super', 'manager'] or not review.answer_review):
328         flash('삭제 권한이 없습니다.')
329         return redirect(url_for(endpoint='cs.review_detail', review_id=review.id))
330
331     db.session.delete(review)
332     db.session.commit()
333
334     flash('삭제되었습니다.')
335     return redirect(url_for('cs.review_list'))

```

이용자 리뷰삭제

```

# 본인 글만 허용
358 if review.user != g.user:
359     flash('본인이 작성한 문의만 확인 가능합니다.')
360     return redirect(url_for('cs.review_list'))
361
362 return render_template(
363     template_name_or_list='cs/review/review_detail.html',
364     review=review
365 )
366

```

이용자 본인ID 글만 사용가능

1:1 관리자 답변 페이지 화면구현

문의유형	영화관	제목	회원	답변상태
강남소프트리조트	영화관	aaa	김동환	답변대기

aaa

(관리자 답변)

답변을 입력하세요.

답변 저장 삭제

Python

```

280 @bp.route(rule='/review/answer/<int:review_id>', methods=('GET', 'POST')) & Y-youngchan
281 @notice_admin_required
282 def review_answer(review_id):
283
284     review = Review.query.get_or_404(review_id)
285
286     if request.method == 'POST':
287
288         review.answer_review = request.form['answer_review']
289         review.answer_create_date = datetime.now()
290         review.answer_status = 'done'
291         review.answer_admin_id = g.user.id
292
293         db.session.commit()
294         flash('답변이 등록되었습니다.')
295
296         return redirect(url_for(endpoint='cs.review_detail', review_id=review.id))
297
298     return render_template(template_name_or_list='cs/review/review_answer.html', review=review)
299

```

관리자 리뷰답변

```

339 @bp.route(rule='/review/detail/<int:review_id>', methods=['GET']) & Y-youngchan +1
340 @login_required
341
342 def review_detail(review_id):
343     review = Review.query.get_or_404(review_id)
344
345     # 로그인 안 했으면 차단
346     if g.user is None:
347         flash('로그인이 필요합니다.')
348         return redirect(url_for('auth.login'))
349
350     # 관리자면 통과
351     if g.user.admin_role in ['super', 'manager']:
352         return render_template(
353             template_name_or_list='cs/review/review_detail.html',
354             review=review
355         )

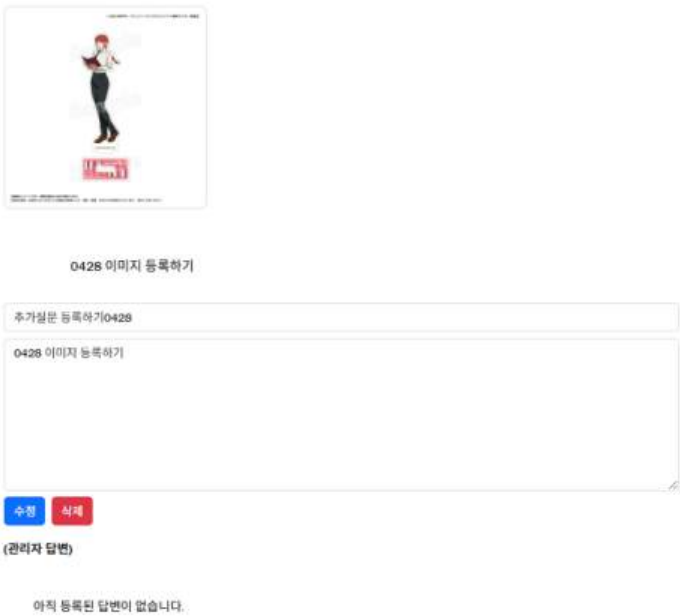
```

문의페이지 안에 관리자면 통과

관리자 계정이면
리뷰페이지에
들어가서 답변을
등록하고 삭제도 가능함

CODE (리뷰 1:1문의 model)

1:1 이용자 문의하는 페이지 화면구현

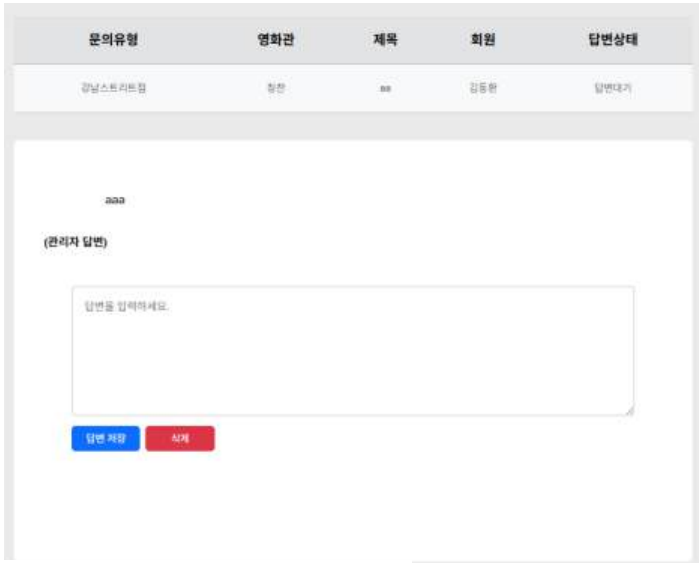


Model

```
211 # 1:1 문의 - review __공자사항
212 class Review(db.Model): 10+ usages Y-youngchan +2
213     id = db.Column(db.Integer, primary_key=True)
214     cs_ask = db.Column(db.String(50), nullable=False)
215     cs_place = db.Column(db.String(50), nullable=False)
216     subject = db.Column(db.String(200), nullable=False)
217     content = db.Column(db.Text(), nullable=False)
218     image_path = db.Column(db.Text())
219     user_id = db.Column(db.Integer, db.ForeignKey('user.id', ondelete="CASCADE"), nullable=False)
220     created_date = db.Column(db.DateTime(), nullable=False)
221     modify_date = db.Column(db.DateTime(), nullable=True)
222     answer_review = db.Column(db.Text(), nullable=True)
223     user = relationship(argument="User", foreign_keys=[user_id])
```

user에 반드시 foreign key가 들어가야 함

1:1 관리자 답변 페이지 화면구현



Modle

```
226 answer_admin_id = db.Column(
227     db.Integer,
228     db.ForeignKey('user.id', ondelete='SET NULL'),
229     nullable=True
230 )
231
232 answer_admin = db.relationship(
233     *args: 'User',
234     foreign_keys=[answer_admin_id],
235     backref='answered_reviews'
236 )
237
238 # 답변 작성일
239 answer_create_date = db.Column(db.DateTime(), nullable=True)
240
241 # 답변 수정일
242 answer_modify_date = db.Column(db.DateTime(), nullable=True)
243
244 # 답변 상태 (대기 / 완료)
245 answer_status = db.Column(
246     db.String(20),
247     nullable=False,
248     default='waiting',
249     server_default='waiting'
250 )
```

관리자가 답변을 작성하면 답변대기에서
답변완료로 바뀐다.

CODE (FAQ)

FAQ 페이지 화면구현

① 자주 찾는 질문 전체가 기본으로 표시됩니다. 원하는 구분의 목록만 보려면 아래 메뉴에서 선택해주세요.

예매 ▼ => 자바스크립트로 필터를 표현함

구분	자주 묻는 질문	등록일
예매	영화 관람시간대 중 조조는 언제인가요?	2026년 04월 22일
조조는 보통 오전 6~10시입니다.		
예매	영화 관람시간대 중 조조는 언제인가요?	2026년 04월 22일

이전 **1** 다음 => 페이지네이션

```

48 # 공지사항 - FAQ
49 class Faq(db.Model):
50     id = db.Column(db.Integer, primary_key=True)
51     kind = db.Column(db.String(100), nullable=False)
52     question = db.Column(db.String(200), nullable=False)
53     answer = db.Column(db.Text(), nullable=False)
54     create_date = db.Column(db.DateTime(), nullable=False)

```

Model

```

158 @app.route("/faq/")
159 def faq_list():
160     page = request.args.get('page', type=int, default=1)
161     kw = request.args.get('kw', default='', type=str)
162     faq_list = Faq.query.order_by(Faq.create_date.desc())
163     faq_list = faq_list.paginate(page=page, per_page=10)
164
165     return render_template("cs/faq/faq.html", faq_list=faq_list)

```

Python

HTML

```

140 <!-- 페이지네이션 -->
141 <nav aria-label="...">
142     <ul class="pagination justify-content-center">
143
144         {% if faq_list.has_prev %}
145         <li class="page-item">
146             <a href="{{ page }}" class="page-link">이전</a>
147         </li>
148         {% else %}
149         <li class="page-item disabled">
150             <a class="page-link" href="javascript:void(0)">이전</a>
151         </li>
152         {% endif %}
153
154         {% for page_num in faq_list.iter_pages() %}
155
156         {% if page_num %}
157
158         {% if page_num != faq_list.page %}
159         <li class="page-item">
160             <a class="page-link" href="{{ page }}">
161                 {{ page_num }}
162             </a>
163         </li>
164         {% else %}
165         <li class="page-item active">
166             <a class="page-link" href="javascript:void(0)">
167                 {{ page_num }}
168             </a>
169         </li>
170         {% endif %}
171
172         {% else %}
173         <li class="page-item disabled">
174             <a class="page-link" href="javascript:void(0)">
175                 ...
176             </a>
177         </li>
178         {% endif %}
179
180         {% endfor %}
181

```

Java Script

```

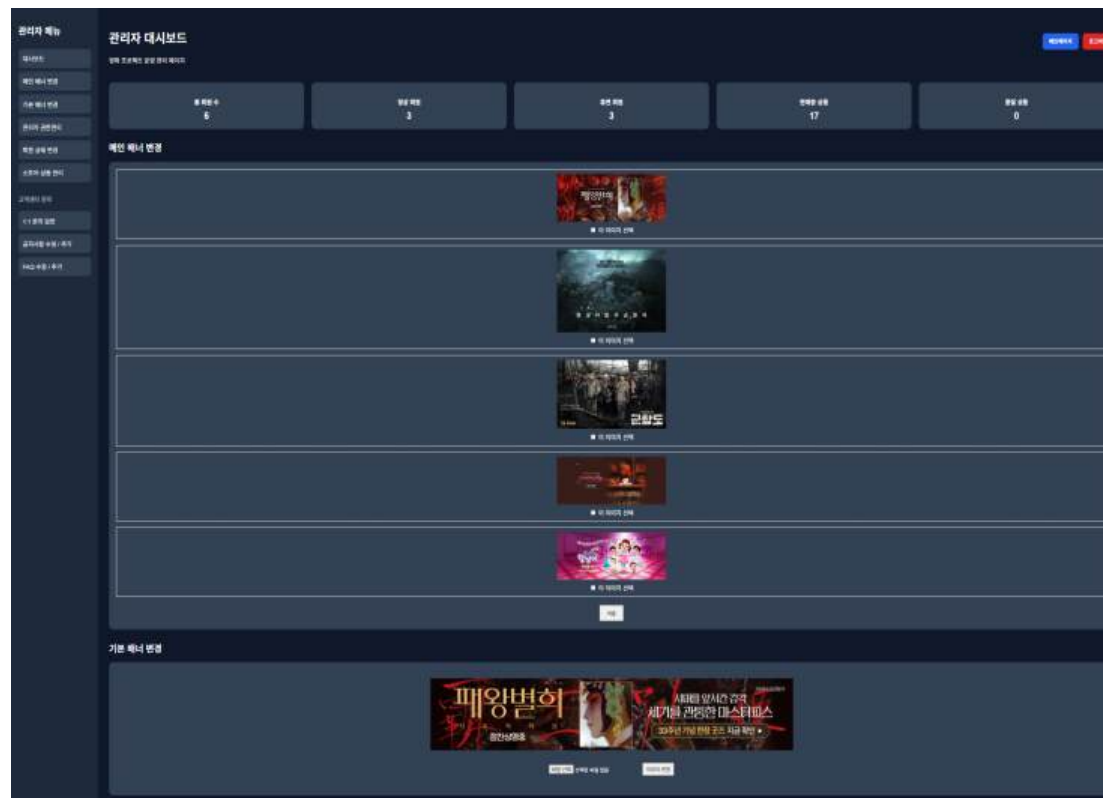
226 // 카테고리 필터
227 document.getElementById("filter").addEventListener("change", function () {
228
229     let selected = this.value;
230     let rows = document.querySelectorAll(".faq-row");
231
232     rows.forEach(function (row) {
233
234         let category = row.querySelector(".category").innerText.trim();
235         let answerRow = row.nextElementSibling;
236
237         if (selected === "전체" || category === selected) {
238             row.style.display = "";
239         } else {
240             row.style.display = "none";
241             answerRow.style.display = "none";
242         }
243     });
244
245 });

```

=> 이전 페이지를 눌러서 더이상 페이지가 없으면 disabled 되어 버튼이 안눌러지게 해줌

CODE (관리자 로그인, 배너관리)

관리자 로그인 시 관리자 페이지 이동 화면 구현
메인 배너 및 기본 배너 변경 기능 구현



Model

```
class User(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    userid = db.Column(db.String(200), unique=True, nullable=False)
    username = db.Column(db.String(150), nullable=False)
    password = db.Column(db.String(200), nullable=False)
    email = db.Column(db.String(120), unique=True, nullable=False)
    phone = db.Column(db.String(20), nullable=False)
    birth = db.Column(db.Date, nullable=False)
    created_at = db.Column(db.DateTime, default=datetime.utcnow)
    Terms_of_Service = db.Column(db.Boolean, nullable=False)
    Privacy_Policy = db.Column(db.Boolean, nullable=False)
    receive_emails = db.Column(db.Boolean, nullable=True, default='False')
    status = db.Column(db.String(20), nullable=False, default='normal', server_default='normal')
    is_admin = db.Column(db.Boolean, nullable=False, default=False, server_default='0')
    admin_role = db.Column(db.String(20), default='none')
```

Python

```
def admin_required(view):
    @functools.wraps(view)
    def wrapped_view(*args, **kwargs):
        # 로그인 안 된 경우 로그인 페이지 이동
        if g.user is None:
            return redirect(url_for('auth.login'))

        # 관리자 계정이 아닌 경우 메인페이지 이동
        if not g.user.is_admin:
            flash('관리자만 접근 가능합니다.')
            return redirect(url_for('main.index'))

        return view(*args, **kwargs)

    return wrapped_view
```

Python

```
@bp.route('/select_main_slider', methods=['POST'])
def select_main_slider():
    order_data = request.form.get('selected_order')

    if not order_data:
        flash("선택된 이미지가 없습니다.", "error")
        return redirect(url_for('auth.admin'))

    selected_ids = order_data.split(',')

    if len(selected_ids) != 3:
        flash("이미지는 3개를 선택해야 합니다.", "error")
        return redirect(url_for('auth.admin'))

    imgs.query.update({'imgs.is_main': False})

    for img_id in selected_ids:
        img = imgs.query.get(int(img_id))
        if img:
            img.is_main = True

    db.session.commit()

    flash("메인 배너가 변경되었습니다.", "success")
    return redirect(url_for('auth.admin'))
```

Python

```
@bp.route('/admin/banner/update/<int:banner_id>', methods=['POST'])
@login_required
@admin_required
def update_banner(banner_id):
    banner = imgs.query.get_or_404(banner_id)

    if banner.img_name not in ['메인 배너1']:
        flash('해당 배너는 수정할 수 없습니다.')
        return redirect(url_for('auth.admin'))

    file = request.files.get('banner_image')

    if not file or file.filename == '':
        flash('파일을 선택하세요.')
        return redirect(url_for('auth.admin'))

    UPLOAD_FOLDER = os.path.join(current_app.root_path, 'static', 'img')
    filename = f"{uuid.uuid4()}_{secure_filename(file.filename)}"
    filepath = os.path.join(UPLOAD_FOLDER, filename)
    file.save(filepath)

    banner.img_url = f'/static/img/{filename}'

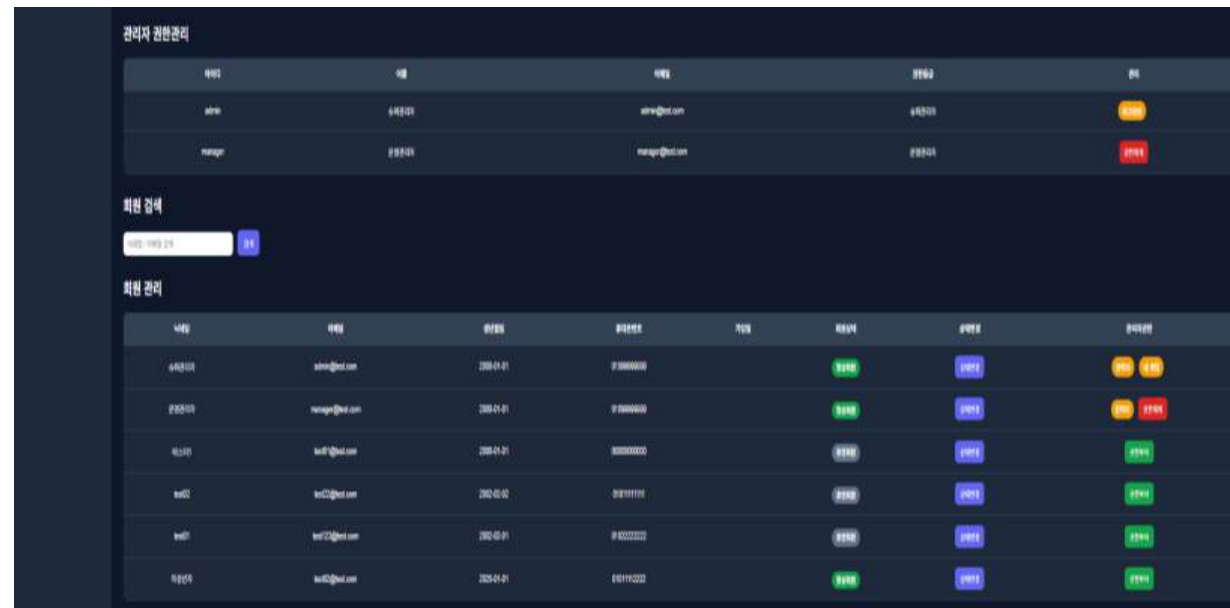
    db.session.commit()

    flash('배너가 변경되었습니다.')
    return redirect(url_for('auth.admin'))
```


CODE (슈퍼관리자 권한)

Python

회원관리 및 관리자 권한 부여 화면구현



```
@bp.route('/admin/user/<int:user_id>/status')
@login_required
@admin_required
def change_user_status(user_id):
    user = User.query.get_or_404(user_id)

    if user.status == 'normal':
        user.status = 'sleep'
    else:
        user.status = 'normal'

    db.session.commit()

    flash('회원 상태가 변경되었습니다.')
    return redirect(url_for('auth.admin'))
```

```
@bp.route('/admin')
@login_required
@admin_required
def admin():
    keyword = request.args.get('keyword', '')

    users = User.query.filter(
        User.username.contains(keyword) |
        User.email.contains(keyword)
    ).all()

    admins = User.query.filter_by(is_admin=True).all()

    products = Product.query.filter(
        Product.Productname.contains(keyword)
    ).all()

    banners = imgs.query.filter(
        imgs.img_name.in_(['메인 배너1'])
    ).all()

    main_images = imgs.query.filter(
        imgs.img_name.like('메인 슬라이더%')
    ).order_by(imgs.id).all()

    selected_ids = session.get('main_slider', [])

    total_users = User.query.count()
    normal_users = User.query.filter_by(status='normal').count()
    sleep_users = User.query.filter_by(status='sleep').count()

    soldout_products = Product.query.filter_by(status='soldout').count()
    normal_products = Product.query.filter_by(status='normal').count()

    return render_template(
        'admin.html',
        users=users,
        admins=admins,
        products=products,
        banners=banners,
        main_images=main_images,
        selected_ids=selected_ids,

        total_users=total_users,
        normal_users=normal_users,
        sleep_users=sleep_users,

        soldout_products=soldout_products,
        normal_products=normal_products
    )
```

```
@bp.route('/admin/user/<int:user_id>/grant-admin')
@login_required
@super_admin_required
def grant_admin(user_id):
    user = User.query.get_or_404(user_id)

    user.is_admin = True
    user.admin_role = 'manager'

    db.session.commit()

    flash('관리자 권한이 부여되었습니다.')
    return redirect(url_for('auth.admin'))
```

```
@bp.route('/admin/user/<int:user_id>/remove-admin')
@login_required
@super_admin_required
def remove_admin(user_id):
    user = User.query.get_or_404(user_id)

    if user.id == g.user.id:
        flash('본인 계정은 권한 해제할 수 없습니다.')
        return redirect(url_for('auth.admin'))

    if user.admin_role == 'super':
        flash('슈퍼관리자는 해제할 수 없습니다.')
        return redirect(url_for('auth.admin'))

    user.is_admin = False
    user.admin_role = None

    db.session.commit()

    flash('관리자 권한이 해제되었습니다.')
    return redirect(url_for('auth.admin'))
```

CODE (관리자 상품 관리)

스토어 상품 판매 상태관리 화면구현

스토어 상품 관리

상품명	종류	가격	재고	상태	관리
101 상품	101	10000	1000	정상	관리
102 상품	102	10000	1000	정상	관리
103 상품	103	10000	1000	정상	관리
104 상품	104	10000	1000	정상	관리
105 상품	105	10000	1000	정상	관리
106 상품	106	10000	1000	정상	관리
107 상품	107	10000	1000	정상	관리
108 상품	108	10000	1000	정상	관리
109 상품	109	10000	1000	정상	관리
110 상품	110	10000	1000	정상	관리
111 상품	111	10000	1000	정상	관리
112 상품	112	10000	1000	정상	관리
113 상품	113	10000	1000	정상	관리
114 상품	114	10000	1000	정상	관리
115 상품	115	10000	1000	정상	관리
116 상품	116	10000	1000	정상	관리
117 상품	117	10000	1000	정상	관리
118 상품	118	10000	1000	정상	관리
119 상품	119	10000	1000	정상	관리
120 상품	120	10000	1000	정상	관리
121 상품	121	10000	1000	정상	관리
122 상품	122	10000	1000	정상	관리
123 상품	123	10000	1000	정상	관리
124 상품	124	10000	1000	정상	관리
125 상품	125	10000	1000	정상	관리
126 상품	126	10000	1000	정상	관리
127 상품	127	10000	1000	정상	관리
128 상품	128	10000	1000	정상	관리
129 상품	129	10000	1000	정상	관리
130 상품	130	10000	1000	정상	관리
131 상품	131	10000	1000	정상	관리
132 상품	132	10000	1000	정상	관리
133 상품	133	10000	1000	정상	관리
134 상품	134	10000	1000	정상	관리
135 상품	135	10000	1000	정상	관리
136 상품	136	10000	1000	정상	관리
137 상품	137	10000	1000	정상	관리
138 상품	138	10000	1000	정상	관리
139 상품	139	10000	1000	정상	관리
140 상품	140	10000	1000	정상	관리
141 상품	141	10000	1000	정상	관리
142 상품	142	10000	1000	정상	관리
143 상품	143	10000	1000	정상	관리
144 상품	144	10000	1000	정상	관리
145 상품	145	10000	1000	정상	관리
146 상품	146	10000	1000	정상	관리
147 상품	147	10000	1000	정상	관리
148 상품	148	10000	1000	정상	관리
149 상품	149	10000	1000	정상	관리
150 상품	150	10000	1000	정상	관리

상품 품절 시 표시 화면구현



HTML

```
<div class="titan_merch">
  {% for product in products if product.Producttype == '굿즈' %}

  <a href="{{ url_for('store.product', id=product.id) }}"
    class="product-card {% if product.status == 'soldout' %}soldout-item{% endif %}">

    {% if product.status == 'soldout' %}
    <div class="soldout-badge">품절</div>
    {% endif %}

  </a>
  </div>
```

```
<section class="table-section" id="product-section">
  <h2>스토어 상품 관리</h2>
  <table>
    <thead>
      <tr>
        <th>상품명</th>
        <th>종류</th>
        <th>가격</th>
        <th>재고</th>
        <th>상태</th>
        <th>상태변경</th>
      </tr>
    </thead>
    <tbody>
      {% for product in products %}
      <tr>
        <td>{{ product.Productname }}</td>
        <td>{{ product.Producttype }}</td>
        <td>{{ product.Productprice }}원</td>
        <td>{{ product.stock }}</td>
        <td>
          {% if product.status == 'normal' %}
          <span class="status normal">판매중</span>
          {% else %}
          <span class="status sleep">품절</span>
          {% endif %}
        </td>
        <td>
          <a href="{{ url_for('auth.change_product_status', product_id=product.id) }}"
            class="btn-change">
            상태변경
          </a>
        </td>
      </tr>
      {% endfor %}
    </tbody>
  </table>
</section>
```

Python

```
@bp.route('/admin/product/<int:product_id>/status')
@login_required
@admin_required
def change_product_status(product_id):
    product = Product.query.get_or_404(product_id)

    if product.status == 'normal':
        product.status = 'soldout'
    else:
        product.status = 'normal'

    db.session.commit()

    flash('상품 상태가 변경되었습니다.')
    return redirect(url_for('auth.admin'))
```


CODE (관리자 공지사항 관련)

관리자 로그인 시 공지사항 화면구현

영화관	제목	등록일
강남스트리트점	강남스트리트점 2026년 12월 휴무일 안내	2026-12
가산디지털점	가산디지털점 2026년 12월 휴무일 안내	2026-12
건대점	건대점 2026년 12월 휴무일 안내	2026-12
용산점	용산점 2026년 12월 휴무일 안내	2026-12
홍대점	홍대점 2026년 12월 휴무일 안내	2026-12
강교점	강교점 2026년 12월 휴무일 안내	2026-12
부천점	부천점 2026년 12월 휴무일 안내	2026-12
동탄점	동탄점 2026년 12월 휴무일 안내	2026-12
수원역점	수원역점 2026년 12월 휴무일 안내	2026-12
송도점	송도점 2026년 12월 휴무일 안내	2026-12
인계점	인계점 2026년 12월 휴무일 안내	2026-12
분당점	분당점 2026년 12월 휴무일 안내	2026-12
양양점	양양점 2026년 12월 휴무일 안내	2026-12
강릉점	강릉점 2026년 12월 휴무일 안내	2026-12
천안점	천안점 2026년 12월 휴무일 안내	2026-12

관리자 계정 공지사항 수정 삭제 버튼 화면구현

영화관	제목	등록일
강남스트리트점	강남스트리트점 2026년 12월 휴무일 안내	2026년 12월 01일 AM 12:00

인녕하세요, 웰컴아트입니다.
먼저 웰컴아트 강남스트리트점을 이용하실 때는 고객님의 항상 깊은 감사 드립니다.

웰컴아트 강남스트리트점은 07스퀘어 건물 전체 의무 휴업일에 따라
12월 9일(화)과 12월 23일(화) 휴관합니다.
그 외 모든 요일은 정상 영업 예정이오니 이용에 참고 부탁드립니다.

※ 의무 휴업 기준: 제 2018-243호 대형마트 및 중대규모점포 의무휴업일 지정 및 영업시간 제한 고시

감사합니다.

수정
삭제

이전 : 서귀포점 2026년 11월 휴무일 안내
다음 : 가산디지털점 2026년 12월 휴무일 안내

목록

Python

```
@bp.route('/notice/create/', methods=('GET', 'POST'))
@notice_admin_required
def notice_create():

    form = NoticeForm()

    if request.method == 'POST' and form.validate_on_submit():

        notice = Notice(
            theater=form.theater.data,
            title=form.title.data,
            content=form.content.data,
            create_date=datetime.now()
        )

        db.session.add(notice)
        db.session.commit()

        flash('공지사항이 등록되었습니다.')

        return redirect(url_for('cs.notice_list'))

    return render_template(
        'cs/notice/notice_form.html',
        form=form
    )
```

CODE (FAQ 관리자 페이지)

관리자 FAQ 등록/수정/삭제 화면 구현

① 자주 찾는 질문 전체가 기본으로 표시됩니다. 원하는 구분의 목록만 보려면 아래 메뉴에서 선택해주세요.

전체 FAQ 등록

구분	자주 묻는 질문	등록일	관리
예매	영화 관람시간대 중 조조는 언제인가요?	2026년 04월 22일	수정 삭제
관람권	특별관 전용 관람권이 따로 있나요?	2026년 04월 22일	수정 삭제
영화관 이용	코즈 상영기준은 어떻게 되나요?	2026년 04월 22일	수정 삭제
예매	영화 관람시간대 중 조조는 언제인가요?	2026년 04월 22일	수정 삭제
관람권	특별관 전용 관람권이 따로 있나요?	2026년 04월 22일	수정 삭제
영화관 이용	코즈 상영기준은 어떻게 되나요?	2026년 04월 22일	수정 삭제

이전 1 다음

Python

```
# FAQ 등록하기
@bp.route('/faq/create/', methods=('GET', 'POST'))
@notice_admin_required
def faq_create():

    if request.method == 'POST':
        faq = Faq(
            kind=request.form['kind'],
            question=request.form['question'],
            answer=request.form['answer'],
            create_date=datetime.now()
        )

        db.session.add(faq)
        db.session.commit()

        flash('FAQ가 등록되었습니다.')
        return redirect(url_for('cs.faq_list'))

    return render_template('cs/faq/faq_form.html', faq=None)
```

```
# FAQ 수정하기
@bp.route('/faq/edit/<int:faq_id>', methods=('GET', 'POST'))
@notice_admin_required
def faq_edit(faq_id):

    faq = Faq.query.get_or_404(faq_id)

    if request.method == 'POST':
        faq.kind = request.form['kind']
        faq.question = request.form['question']
        faq.answer = request.form['answer']

        db.session.commit()

        flash('FAQ가 수정되었습니다.')
        return redirect(url_for('cs.faq_list'))

    return render_template('cs/faq/faq_form.html', faq=faq)

# FAQ 삭제하기
@bp.route('/faq/delete/<int:faq_id>')
@notice_admin_required
def faq_delete(faq_id):

    faq = Faq.query.get_or_404(faq_id)

    db.session.delete(faq)
    db.session.commit()

    flash('FAQ가 삭제되었습니다.')

    return redirect(url_for('cs.faq_list'))
```


CODE (각종 Seed Data)

```
def save_movies():
    from pybo.models import Movie, db

    url = "https://api.themoviedb.org/3/movie/now_playing"
    params = {
        "api_key": "85a24c607dac1e0d9139a903ee0f509b",
        "language": "ko-KR",
        "region": "KR"
    }

    res = requests.get(url, params=params)
    data = res.json()

    for m in data['results']:
        if not Movie.query.filter_by(tmdb_id=m['id']).first():

            certification = get_certification(m['id'])
            genres = get_genres(m['genre_ids'])
            actors = get_actors(m['id'])

            movie = Movie(
                tmdb_id=m['id'],
                title=m['title'],
                overview=m['overview'],
                poster_path=m['poster_path'],
                release_date=m['release_date'],
                vote_average=m['vote_average'],
                certification=certification,
                genres=genres,
                actors=actors
            )
            db.session.add(movie)
```

Movie Seed Data

```
def seed_admin():
    admins = [
        {
            "userid": "admin",
            "username": "슈퍼관리자",
            "password": "1111",
            "email": "admin@test.com",
            "phone": "01099999999",
            "birth": "2000-01-01",
            "role": "super"
        },
        {
            "userid": "manager",
            "username": "운영관리자",
            "password": "1111",
            "email": "manager@test.com",
            "phone": "01099999999",
            "birth": "2000-01-01",
            "role": "manager"
        }
    ]

    user = User(
        userid=data["userid"],
        username=data["username"],
        password=generate_password_hash(data["password"]),
        email=data["email"],
        phone=data["phone"],
        birth=date(2000, 1, 1),
        Terms_of_Service=True,
        Privacy_Policy=True,
        receive_emails=False,
        status='normal',
        is_admin=True,
        admin_role=data["role"]
    )

    db.session.add(user)
```

Admin Seed Data

```
def insert_test_data(n=6):
    """테스트용 질문 데이터 n개 생성"""
    app = create_app() # Flask 컨텍스트 가 필요
    with app.app_context():
        kinds = ["영화관 이용", "관람권", "예매", "영화관 이용", "관람권", "예매"]
        questions = ['굿즈 상영기준은 어떻게 되나요?', '특별관 전용 관람권이 따로 있나요?', '영화 관람시간대 중 조조는 언제인가요?', '굿즈 상영기준은 어떻게 되나요?',
                    '특별관 전용 관람권이 따로 있나요?', '영화 관람시간대 중 조조는 언제인가요?']
        answers = [
            "굿즈 상영 기준은 영화사 정책에 따라 다릅니다.",
            "특별관 전용 관람권은 별도 구매가 필요합니다.",
            "조조는 보통 오전 6~10시입니다.",
            "굿즈 상영 기준은 영화사 정책에 따라 다릅니다.",
            "특별관 전용 관람권은 별도 구매가 필요합니다.",
            "조조는 보통 오전 6~10시입니다."
        ]

        for i in range(n):
            q = Faq(
                kind=kinds[i],
                question=questions[i],
                answer=answers[i],
                create_date=datetime.now()
            )
            db.session.add(q)
            db.session.commit()
            print(f"{n}개의 테스트 데이터가 생성되었습니다.")
```

FAQ Seed Data

```
def insert_test_data(n=10):
    """테스트용 질문 데이터 n개 생성"""
    app = create_app()
    with app.app_context(): # Flask 컨텍스트 필요
        for i in range(n):
            q = Review(
                question='테스트 데이터 입니다:[%03d]' % i,
                info = '답변작성',
                content='내용없음',
                create_date=datetime.now()
            )
            db.session.add(q)
            db.session.commit()
            print(f"{n}개의 테스트 데이터가 생성되었습니다.")
```

Review Seed Data

```
if __name__ == "__main__":
    insert_test_data()
```


CODE (각종 Seed Data)

```
with app.app_context():
    imgs.query.delete()
```

Images Seed Data

```
data = [
    imgs(img_name='메인 로고1', img_url='/static/img/filmatique_white.png', img_type='main'),
    imgs(img_name='메인 로고2', img_url='/static/img/filmatique_red.png', img_type='main'),
    imgs(img_name='메인 슬라이더1', img_url='/static/img/main_movie_slide_1.jpg', img_type='main'),
    imgs(img_name='메인 슬라이더2', img_url='/static/img/concrete.jpeg', img_type='main'),
    imgs(img_name='메인 슬라이더3', img_url='/static/img/The_Battleship_Island.jpg', img_type='main'),
    imgs(img_name='메인 슬라이더4', img_url='/static/img/main_movie_slide_2.png', img_type='main'),
    imgs(img_name='메인 슬라이더5', img_url='/static/img/main_movie_slide_3.jpg', img_type='main'),
    imgs(img_name='메인 이벤트1', img_url='/static/img/event1.jpg', img_type='main'),
    imgs(img_name='메인 이벤트2', img_url='/static/img/event2.jpg', img_type='main'),
    imgs(img_name='메인 이벤트3', img_url='/static/img/event3.jpg', img_type='main'),
    imgs(img_name='메인 이벤트4', img_url='/static/img/event4.jpg', img_type='main')
]
```

```
with app.app_context():
```

```
screens = Screen.query.all()
```

```
if not screens:
    print("Screen 데이터 없음")
    exit()
```

```
rows = ['A', 'B', 'C', 'D', 'E', 'F']
cols = range(1, 11) # 1~10
```

```
for screen in screens:
    print(f"{screen.name} 좌석 생성 중...")
```

Seats Seed Data

```
for row in rows:
    for col in cols:
```

```
# 중복 체크
exists = Seat.query.filter_by(
    screen_id=screen.id,
    row=row,
    col=col
).first()
```

```
if exists:
    continue
```

```
seat = Seat(
    screen_id=screen.id,
    row=row,
    col=col
)
```

```
def seed_products():
```

Products Seed Data

```
products = [
    {"id": 1, "Productname": "일반 관람권(2D)", "Producttype": "티켓", "Productprice": 18000, "stock": 1000,
     "Productdescription": "일반 관람권(2D) 1매", "Productimage_url": "/static/img/redticket.png", "Productlimit": 10, "Productdate": "온라인관람권 24 개월"},

    {"id": 2, "Productname": "VIP 관람권", "Producttype": "티켓", "Productprice": 50000, "stock": 1000,
     "Productdescription": "VIP 관람권 1매", "Productimage_url": "/static/img/blackticket.png", "Productlimit": 2, "Productdate": "온라인관람권 24 개월"},

    {"id": 3, "Productname": "BEST COMBO 교환권", "Producttype": "스낵", "Productprice": 14500, "stock": 1000,
     "Productdescription": "반반콤보 or 싱글스낵콤보 중 택 1", "Productimage_url": "/static/img/BEST COMBO 교환권.png", "Productlimit": 10, "Productdate": "스위트샵 상품권 24 개월"},

    {"id": 4, "Productname": "SINGLE COMBO 교환권", "Producttype": "스낵", "Productprice": 10000, "stock": 1000,
     "Productdescription": "싱글커피/내맘대로콤보 중 택 1", "Productimage_url": "/static/img/SINGLE COMBO 교환권.png", "Productlimit": 10, "Productdate": "스위트샵 상품권 24 개월"},

    {"id": 5, "Productname": "SNACKS 교환권", "Producttype": "스낵", "Productprice": 6500, "stock": 1000,
     "Productdescription": "팝콘치킨/소떡소떡(부링글, 맵스터) 중 택 1", "Productimage_url": "/static/img/SNACKS 교환권.png", "Productlimit": 10, "Productdate": "스위트샵 상품권 24 개월"},

    {"id": 6, "Productname": "팝콘(L) 교환권", "Producttype": "스낵", "Productprice": 8500, "stock": 1000,
     "Productdescription": "고소/달달한 빅사이즈 팝콘", "Productimage_url": "/static/img/popcorn_L.png", "Productlimit": 10, "Productdate": "스위트샵 상품권 24 개월"},

    {"id": 7, "Productname": "팝콘(M) 교환권", "Producttype": "스낵", "Productprice": 7500, "stock": 1000,
     "Productdescription": "고소/달달한 기본사이즈 팝콘", "Productimage_url": "/static/img/popcorn_M.png", "Productlimit": 10, "Productdate": "스위트샵 상품권 24 개월"},
]
```


CODE (각종 Seed Data)

Notice Seed Data

```
def insert_test_data(n=12):
    """테스트용 질문 데이터 n개 생성"""
    app = create_app() # Flask 컨텍스트 가 필요
    with app.app_context():
        years=[2025,2026]
        theaters=["강남스트리트점", "가산디지털점", "건대점", "용산점", "홍대점", "광교점", "부천점",
        "대구점", "포항점", "경주점", "울산점", "통영점", "김해점", "부산갈매기점", "제주점", "서귀포점"]

        for year in years:
            for i in range(n):
                for theater in theaters:
                    q = Notice(
                        theater=theater,
                        title=f'{theater} {year}년 {i + 1}월 휴무일 안내',
                        content=f'''안녕하세요, 필름아티크입니다.

먼저 필름아티크 {theater}을 이용해주시는 고객님들께 항상 깊은 감사 드립니다.

필름아티크 {theater}은 67스퀘어 건물 전체 의무 휴업일에 따라
{i+1}월 9일(월)과 {i + 1}월 23일(월) 휴관합니다.
그 외 모든 요일은 정상 영업 예정이오니 이용에 참고 부탁드립니다.

※ 의무 휴업 기준 : 제 2018-243호 대형마트 및 준대규모점포 의무휴업일 지정 및 영업시간 제한 고시

감사합니다.'''

                    created_date=datetime(year, i+1, 1)
                    db.session.add(q)
```

```
THEATERS_BY_REGION = {
    "서울": ["강남스트리트점", "가산디지털점", "건대점", "용산점", "홍대점"],
    "경기/인천": ["광교점", "부천점", "동탄점", "수원역점", "송도점", "인계점", "분당점"],
    "강원": ["양양점", "강릉점"],
    "충청/대전": ["천안점", "오송점", "대전성심당점", "논산훈련소점"],
    "전라/광주": ["광주점", "익산점", "전주점"],
    "경북/대구": ["대구점", "포항점", "경주점"],
    "경남/부산": ["울산점", "통영점", "김해점", "부산갈매기점"],
    "제주": ["제주점", "서귀포점"],
}
```

```
SCREENS_PER_THEATER = 3
MOVIES_PER_SCREEN = 20
DAYS_TO_SEED = 7 # 7일치 상영시간 생성
MOVIE_DURATION_MINUTES = 100
BREAK_MINUTES = 10
START_HOUR = 8
```

Schedule Seed Data

```
def seed_schedules():
    movies = Movie.query.order_by(Movie.id).limit(MOVIES_PER_SCREEN).all()
    screens = Screen.query.order_by(Screen.id).all()

    base_date = datetime.now().replace(hour=START_HOUR, minute=0, second=0, microsecond=0)

    schedule_count = 0

    for screen in screens:
        for day_offset in range(DAYS_TO_SEED):

            # 하루 시작
            day_start = base_date + timedelta(days=day_offset)

            # 하루 마감 (다음날 02:30)
            day_end = (day_start + timedelta(days=1)).replace(hour=2, minute=30, second=0, microsecond=0)

            current_start = day_start

            movie_index = 0

            while current_start < day_end:

                movie = movies[movie_index % len(movies)]

                start_time = current_start
                end_time = start_time + timedelta(minutes=MOVIE_DURATION_MINUTES)

                # 끝나는 시간이 영업시간 넘으면 종료
                if end_time > day_end:
                    break

                existing = Schedule.query.filter_by(
                    screen_id=screen.id,
                    start_time=start_time
                ).first()
```

Review

01

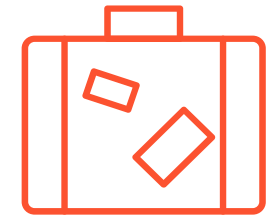
강민식



이번 프로젝트를 진행하면서 초반에 진행 방향을 잘못 잡아 큰 어려움이 있었지만 그럼에도 팀원들의 노력으로 방향을 다시 잡고 무리 없이 진행을 했다. 그리고 작업 중에 예매가 가능한 시스템을 구현하는 데 있어서 어려움은 있었지만 흥미를 느낄 수 있었고 초반에 방향을 잘 잡고 진행을 했다면 성인인증이나 여러 시스템을 추가할 수 있었을 것 같아 아쉬웠다.

02

김동환



초기에는 하드코딩으로 기능 흐름을 먼저 구현하면서 전체 서비스 구조를 빠르게 이해할 수 있었고, 이후 DB 연동 과정에서 데이터 설계의 중요성을 체감했다. 특히 회원정보와 결제 데이터를 다루면서 데이터 무결성과 보안 처리의 필요성을 직접 경험할 수 있었다.

03

유영찬



요즘 AI 기술의 발전으로 인해 정말 많은 정보와 다양한 도움을 받을 수 있다는 사실이었다. 아이디어 정리, 코드에 발생한 오류 문제 해결 과정에서 AI를 활용하니 시간 효율이 높아졌고, 이전보다 더 빠르고 원하는 기획에 맞게 프로젝트를 진행할 수 있었다.

04

조예연



프로젝트를 끝내면서 데이터베이스가 얼마나 중요한지, 그리고 내가 얼마나 부족한지 깨달았다. ChatGPT에 너무 의존한 것 같아 나중에 꼭 내 자신을 점검해야겠다는 필요성을 느꼈다. 마지막으로, 내가 직접 데이터베이스를 만든다는 점이 매우 신기하게 느껴졌다.

감사합니다.

매운짬뽕

